

# US Patent & Trademark Office

## Patent Public Search | Text View

---

United States Patent Application Publication

20250278644

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

Thomas; James et al.

---

## CONTEXTUAL SEMANTIC DERIVATION OF DATA RELATIONSHIPS

---

### Abstract

The present invention includes novel methods and systems for deriving meaning from context, enabling the automation of processes that currently require significant human judgment and intervention. An autonomous event-driven system runs on a continuous basis over time, detecting and responding to new events as new information is obtained (including the mere passage of time) to implement virtually any scenario in which relationships among data within and across documents are difficult to discern (without human intervention) from the explicit information contained in the documents (DDRs). Trained models perform contextual semantic derivation (CSD) to derive meaning from context within and across documents in the form of DD Rs and other relationships stored in an iteratively updated knowledge graph, which is leveraged to perform lower-level document processing tasks (capture, classification, matching, reconciliation, etc.) as well as higher-level tasks (natural-language interrogation, anomaly detection and resolution, decisioning and analytics).

---

**Inventors:** Thomas; James (Wilmington, DE), Markle; Stephen (Laguna Niguel, CA), Courtade; Christopher Ryan (Dayton, KY)

**Applicant:** Itemize Corporation (Wilmington, DE)

**Family ID:** 96881511

**Assignee:** Itemize Corporation (Wilmington, DE)

**Appl. No.:** 19/010696

**Filed:** January 06, 2025

### Related U.S. Application Data

us-provisional-application US 63559369 20240229

---

## Publication Classification

Int. Cl.: G06N5/022 (20230101); G06N5/04 (20230101)

U.S. Cl.:

CPC G06N5/022 (20130101); G06N5/041 (20130101);

---

## Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application claims priority to U.S. provisional patent application Ser. No. 63/559,369, filed Feb. 29, 2024 and entitled “Contextual Semantic Derivation of Data Relationships,” the disclosure of which is hereby incorporated by reference as if fully set forth herein.

### BACKGROUND

#### Field of Art

[0002] The present invention relates generally to the automation of personal, business and other processes, and more particularly to the automation of processes, such as those inherent in financial transactions, in which relationships among data within and across documents are difficult to discern (without human intervention) from the explicit information contained in those documents.

#### Description of Related Art

[0003] Despite the dramatic impact of computerization on our personal and business lives over the past century, the myth of the “paperless office” remains just that—a myth describing a goal that may well never become reality. For a variety of reasons, paper is likely to be here to stay for the foreseeable future.

[0004] Moreover, even if “electronic documents” ever completely replace paper, even these documents often reveal an incomplete picture of the underlying “narrative” or “story” of the transactions and related processes spawning such documents. For example, such documents are far from error-free (e.g., due to data entry errors), and relationships among data within and across such documents are often not self-evident (e.g., where a missing or incorrect vendor name must be inferred from other relevant “matching” information across multiple documents).

[0005] As will become apparent, these “difficult-to-derive relationships” (or “DDR”s) are the source of many problems in which “hidden” context results in a failure to derive meaning. These problems currently are “resolved” by an overreliance on human judgment and intervention.

[0006] It is therefore not surprising that, despite the ever-increasing level of computerization and automation of certain aspects of a process, human judgment remains not only a necessary but a pervasive component of the implementation of virtually any non-trivial process. Eliminating human judgment entirely may well be a fool's errand. But, as will become evident, there currently exist many opportunities to automate processes that are currently relegated to human judgment and intervention due to the difficulty of deriving meaning from context.

[0007] While much of the following discussion relates to financial business transactions, it will become apparent that the concepts discussed herein apply to other types of personal, business, educational, governmental and other processes, including those outside the financial realm.

Virtually any process that currently involves interrelated “documents” (paper, electronic or otherwise)—in which relationships among data within and across such documents are difficult to discern from the explicit information contained in such documents—presents similar problems that are currently addressed via frequent human judgment and intervention.

[0008] For example, various non-financial scenarios involve DDRs. Consider a college admissions

officer examining applicants' transcripts, test scores, personal essays and other application details, and employing human judgment to determine which applicants merit admission. Or a medical doctor examining patients' current and historical test results, examination and treatment records and other data, and similarly employing human judgment to form diagnoses of patient conditions and prognoses for the future progression of such conditions. In each of these scenarios, human judgment is employed in part due to the difficulty of deriving relationships among data within and across documents (i.e., DDRs) despite access to the explicit information contained within such documents.

[0009] Moreover, the concept of “documents” as described herein is more expansive than the characters and words found in typical paper or electronic documents or files. It covers not only the locations of characters and words on a page and the structure of such data (e.g., in a database or spreadsheet file), but virtually any other medium, metadata or other attribute of information from which “signal” can be distinguished from “noise” to reveal meaning.

[0010] For example, consider audio from which one's tone of voice can be discerned in addition to spoken words, permitting inferences of meaning, such as the relative importance of particular information. A phone conversation with an agitated key supplier regarding an overdue invoice might result in the escalation of that payment request (as contrasted with other similar requests). Similarly, a customer service agent might discern far more understanding of a customer's problem from a phone call than from a text chat.

[0011] Humans frequently make such inferences. But automated systems currently emphasize “lower dimensions” of information, rather than more abstract relationships among data that enable more accurate distinctions between signal and noise to derive meaning. Not unlike the book *Flatland* [“*Flatland: A Romance of Many Dimensions*,” Edwin A. Abbott (**1884**)], in which physical dimensions limit one's ability to discern reality, so too can the examination of more abstract dimensions of documents reveal meaning to a far greater extent.

[0012] For example, one can think of characters and words on a page as lower dimensions of documents, with higher dimensions including structured data, audio and other media beyond text, the concept of time and ultimately any attribute of data from which signal can be distinguished from noise. These higher dimensions reveal meaning via the relationships among data within and across documents. As will become apparent, automated techniques such as optical character recognition (OCR) focus primarily on these lower dimensions.

[0013] Regardless of the particular use case or scenario, there exist basic “tasks” that must be performed to implement that scenario. For example, even within the context of financial business transactions, a vast array of vertical use cases share similar “business process automation” problems that are currently addressed all too frequently via human judgment and intervention. Accounts payable (AP) and accounts receivable (AR) departments struggle with the management of purchase orders (POs), invoices, checks, receipts, vendor statements, title documents and a host of other document types.

[0014] In addition to common transactions involving POs and invoices representing the sale of goods and services, other scenarios include mortgage appraisals and lending, contractor bonding, vehicle title transfers, chargebacks for disputed charges and calculation of rebates, among many others. While the specific tasks and document types may differ across different scenarios, there remains a great deal of overlap with respect to common tasks that present obstacles to automation (again due at least in part to the difficulty of deriving relationships among data within and across documents to “fill in the gaps” and resolve inconsistencies).

[0015] For example, consider the “wholesale lockbox”—a common service typically provided by banks or other financial institutions in an effort to simplify the “ingestion” of receivables by a company and facilitate its cash-flow management. In this scenario, customer payments are redirected from the company's address to a bank or other intermediary that processes the payments, extracts relevant information (e.g., customer name, payment amount, account number, etc.) and

deposits the payments in the company's accounts along with reports of varying degrees of detail. [0016] Similar “retail lockbox” services provide essentially the same functionality, but in a B2C (business-to-consumer) rather than a B2B (business-to-business) context. For example, utility companies with hundreds of thousands or millions of customers making monthly payments often rely on retail lockbox services to manage and process such payments.

[0017] Depending on the scenario, these lockbox services involve a variety of different document types, including invoices, remittance documents, envelopes, checks and vendor statements, among others. Moreover, each document includes multiple different “document components” or fields of data. The intermediary bank must process these paper and electronic documents as they arrive, despite the fact that such documents often fail to conform to predefined templates or formats.

[0018] While these lockbox services provide many advantages to companies, they are currently fraught with problems, many of which result in an overreliance on human intervention. For example, despite the proliferation of electronic payments, consumers and businesses alike still employ paper and electronic checks as a form of payment. The intermediary bank must still perform or delegate the tasks of opening envelopes, reading, interpreting and entering data into a computer and/or relying on OCR technology to scan documents.

[0019] It is therefore not surprising that one major problem with existing lockbox services (and a vast array of other financial business transactions, as well as non-financial scenarios) is limited data accuracy regarding the “capture” of information. Handwritten and even electronic checks frequently contain errors in names, dates, amounts, etc. Such errors are compounded by OCR technology, which performs particularly poorly with low-quality images, and even moreso by human data entry operators who must interpret what they see in a document and attempt to translate that into keystrokes (while often under immense pressure to process vast amounts of data within a limited period of time).

[0020] Another major problem lies in the inherent need to “classify” various different document types, while also recognizing individual document fields. This process is currently a manual one, for the most part, involving human judgment regarding the relationships of data within and across documents. Failure to properly classify documents and their component fields, and recognize relationships among them (DDRs in particular), is primarily the result of human error in failing to infer meaning from contextual clues within and across documents.

[0021] Many of these errors go uncorrected, while others are detected and corrected only as a result of human intervention. While auto-detection and correction technologies exist to a limited extent, they typically are restricted to predefined document layouts and formats, which may be too limiting for many scenarios.

[0022] As will become apparent, these “capture” and “classification” problems exist not only in the context of lockbox services, but across the spectrum of financial business transactions and beyond. What these scenarios share in common is that information must be captured with sufficient accuracy, and various document types and components must be classified as prescribed by the requirements of a particular scenario.

[0023] It should be noted that one common yet particularly difficult problem (with regard to capture, classification and many other tasks) results from scenarios lacking predefined document layouts and formats. For example, both electronic files and written pages of paper often contain multiple documents that are not clearly delineated from one another. A single page of paper may contain two invoices and a purchase order stemming from three different transactions. Or a single invoice may span multiple pages. Similarly, a single electronic file may contain multiple documents across multiple document types, only some of which relate to the same transaction.

[0024] The failure to accurately split, merge or otherwise “segment” paper and electronic files to identify distinct documents and component fields is yet another reason companies are often forced to rely on human judgment and intervention, despite their desire for fully automated solutions.

[0025] In addition to capture and classification problems, data must also be “matched” within and

across documents. For example, payments must be matched to their original invoices. It should be noted that this matching problem is not limited to one-to-one relationships, such as the matching of a single payment to its corresponding invoice.

[0026] For example, a single payment could apply to multiple invoices, or conversely a single invoice might be satisfied via multiple payments. Such many-to-many relationships are often difficult to discern from the explicit information contained within one or more documents, and therefore currently remain undetected, at least without extensive human intervention.

[0027] Such relationships also occur not only at a “document” level, but also at lower levels of abstraction, such as document components, fields and line-item data of varying detail. For example, in the context of credit card transactions, level 1 data may refer to a merchant name, billing zip code and transaction amount. Level 2 data adds fields such as sales tax amounts, tax IDs and merchant state codes, among others. Level 3 data adds even more detailed information, such as order numbers, ship to/from zip codes, item descriptions and quantity, and a host of other data fields.

[0028] The existence of numerous fields not only increases the difficulty of maintaining an acceptable level of data accuracy, but also exacerbates the matching problem, as the potential mismatches between and among different data fields increases exponentially. For example, a merchant name on an invoice might not match the merchant name on a corresponding remittance document (or may not have been detected due to its unusual placement within a document). Item quantity mismatches may only be revealed when adding the quantities across multiple different shipments (another task which often occurs only as a result of human judgment and intervention).

[0029] In addition to the myriad instances of “mismatches,” companies must also eventually “reconcile” such mismatches, in part by determining whether there are “good” or “bad” reasons for the mismatch. For example, a supplier that ships items only by the dozen might ship 12 units in response to an order for 10 units (arguably the correct result in that circumstance). Or a remittance document might reflect payment for only 17 tomatoes, despite an order for 20 tomatoes, because 3 tomatoes were “rotten” upon arrival (again arguably the correct result). But if a supplier ships only 10 items in response to an order for 20 items because the supplier only has 10 items in stock, this result might be deemed a “bad” or incorrect result, requiring a future shipment of the remaining items, and perhaps even a financial penalty.

[0030] In short, “matching” problems are often accompanied by related “reconciliation” problems, both of which currently result in many errors, as well as increased cost due to the need for human judgment and intervention. And human intervention only partly addresses these problems due to the prevalence of human error.

[0031] It is apparent that errors can result from many aspects of the overall process in almost any non-trivial use case scenario—from data capture errors to improper classification of document types or component fields to an endless array of matching and reconciliation errors.

[0032] Human judgment and intervention, while reducing some of these errors, also contributes additional time, cost and human error to the problem.

[0033] Further errors result from a lack of “verification”—another task that currently either fails to occur, or requires extensive human judgment and intervention. For example, data within a Tax ID field may have a valid format, but may not actually contain the correct value in context. Or the value-added tax (VAT) charged by a business in a particular country may appear on the surface to be valid, but may actually be the result of applying the wrong VAT rate applicable to that country.

[0034] Moreover, the necessity for “compliance” with particular laws and regulations, as well as scenario-specific rules, often requires a contextual understanding of such rules that cannot easily be discerned from the explicit information contained within and across available documents. For example, consider a rule that limits the cumulative value of monthly transactions. Even an automated application of that rule may require access to all relevant transactions within a particular month. But if a date is missing from a particular document, and can only be inferred from

information in other documents, a lack of compliance may well go unnoticed, or be detected and corrected, if at all, only as a result of human judgment and intervention.

[0035] Similar errors result from inaccurate “coding” of data within document component fields, such as standard accounting codes or custom company-specific fields. On the surface, automatically detecting incorrect codes may appear to be relatively simple. But when the format of a substantively incorrect code is accurate, such detection becomes more complex and resistant to automation, as it often requires contextual clues within and across documents. Substantive errors are in essence “hidden” by accurate data formats, resulting in reliance on human judgment and intervention.

[0036] It should be noted that, as documents are captured, classified, matched, reconciled, verified and coded, most scenarios involve other higher-level tasks (beyond basic “document processing”) that are performed with respect to such documents. For example, scenarios typically involve one or more ultimate “decisions” (e.g., the approval or rejection of a loan application) as well as component interim “actions” (e.g., the approval of expenditures exceeding a specified minimum amount). Such “decisioning” tasks are a central part of the workflow (automated or otherwise) prescribed by most scenarios.

[0037] In addition, individuals within a company are often tasked with performing various “analytics” with respect to transactions and their associated documents. These tasks are typically instigated by people formulating queries to a database that is populated manually as documents are processed. Another extremely important higher-level task involves “interrogation” or the formulation of queries that are typically submitted to the staff of AP, AR and other company departments.

[0038] These higher-level tasks (decisioning, analytics and interrogation) also represent areas that frequently involve human judgment and intervention. The current practice of “querying databases” is inherently limiting, as it typically involves manually predetermined relationships among data, as opposed to relationships that are derived automatically on an iterative ongoing basis as information is obtained.

[0039] While these various document-processing and higher-level tasks are common to a wide variety of financial and other scenarios, they all present problems that are particularly resistant to automation. Raw information itself may be inaccurate. Document formats may vary greatly. More significantly, errors and other “anomalies” in data interrelationships (both within and across documents) are often difficult to discern (without human intervention) from the explicit information contained within documents.

[0040] Current overreliance on human judgment and intervention presents its own problems, such as introducing further “human” errors into the process while increasing both time and expense. For example, the ability to train and scale a large number of people to address these types of errors presents logistical, time and cost concerns, as well as security and regulatory compliance concerns. Moreover, reliability, consistency and data integrity are also often compromised by human error. Not every vendor addresses these issues to the same extent.

[0041] Some may encounter customer resistance due to their lack of particular types of exception handling, or their rigidity in requiring predefined document templates or formats.

[0042] A major reason for this resistance to automation and overreliance on human judgment and intervention is the difficulty of deriving intended meaning from context (both within and across documents). What humans provide is the ability to extract context from the explicit information contained within documents to identify and address anomalies.

[0043] There is thus a need to automate this process of “contextual semantic derivation” (referred to herein as “CSD”) to derive intended meaning from the explicit information contained within documents while reducing substantially the current overreliance on human judgment and intervention (and its associated introduction of additional errors, time and expense). Such an automated process would need to identify, as well as address, anomalies that constitute DDRs—i.e.,

data interrelationships (both within and across documents) that are difficult to discern from the explicit information contained within documents.

[0044] Efforts to address these problems, within and beyond the realm of financial business transactions, have been limited at best. The emergence of machine learning (ML) and other forms of AI has been recognized as a key factor in resolving various aspects of these problems—due to its ability to recognize patterns, learn from past behavior and predict likely outcomes.

[0045] In the context of financial transactions, many companies tout their use of “AI-based” technologies to address these problems. Yet, virtually all such “solutions” are limited to isolated specific tasks, such as fraud detection and classification of predefined document types. These approaches do not encompass the many varied yet related tasks present in any non-trivial scenario. And companies typically must decide when and how frequently to invoke this AI-based functionality.

[0046] One overarching problem is that many of the tasks involved in any particular scenario are interrelated, causing an error with respect to one task (e.g., matching data fields across documents) to cascade into errors regarding other tasks (e.g., authorizing a decision to pay an invoice, generating a summary report, providing responses to natural language queries, etc.). Interdependencies among various tasks create recurring problems in determining when and how particular tasks should be performed.

[0047] Failure to perform these various tasks on an ongoing basis (e.g., as new information is obtained, including the mere passage of time) causes many errors to go undetected or be compounded as a result of faulty assumptions. For example, a document may remain unclassified or be improperly classified due to missing or contradictory information, such as a missing date or a misspelled vendor name. Subsequent information from a new email or other related document could enable the original document to be classified properly—but only if that fact is recognized in context and the classification task is “reinvoked.”

[0048] Moreover, the mere passage of time may constitute an “event” requiring that particular tasks be repeated. For example, recognizing that an invoice has not been received for an unusually long period of time could enable the reinvocation of a reconciliation task which might result in a reminder email being sent to a supplier as the close of the fiscal quarter approaches. The absence of a document or other information is itself an event that may suggest further inquiry.

[0049] It should be noted that humans routinely perform tasks in response to various “events” including the receipt of new information, the passage of time, the absence of an expected document or other information, and other metadata or information that suggests an anomalous event has occurred or some other event is anticipated to occur in the near future.

[0050] Existing automated approaches rarely anticipate upcoming events (e.g., a payment deadline or a potential overdrawn account), much less take proactive steps to resolve them. To address the problems described above, it is important that any solution be capable of detecting events of various types and taking proactive steps in response, including the performance or reinvocation of particular tasks.

[0051] Apart from these “vertical solutions” in the field of financial business transactions, companies such as Google have developed more general, “technology-centric” approaches to these problems. For example, Google’s “Document AI” technology employs AI-based technologies in an effort to “gain deeper insights from unstructured or structured document information.” While Google employs an “Enterprise Knowledge Graph” to organize information, it relies on customers to utilize its Entity Reconciliation and other APIs to populate its knowledge graph.

[0052] In other words, Google’s Document AI technology does not iteratively populate its Enterprise Knowledge Graph with relationships that represent meaning derived from processing documents as information is obtained (e.g., in the context of the various interrelated tasks involved in a particular use-case scenario—such as capture, classification, matching, reconciliation, verification, coding, decisioning, analytics and interrogation, among others). Moreover, it does not

employ the current state of its Enterprise Knowledge Graph to perform such tasks on an ongoing basis, much less recognize when such tasks should be performed or repeated.

[0053] Instead, Google's Document AI technology requires its customers to utilize Google's Entity Reconciliation, Google Knowledge Graph Search and other APIs to populate and search the Enterprise Knowledge Graph, which essentially represents the relationships already present in the customers' own files. In other words, customers must determine when to invoke particular tasks to derive meaning from context, how and when to update a knowledge graph (or other means of reflecting current relationships among data within and across documents) to reflect changes resulting from the performance of such tasks, and when to repeat such tasks (e.g., in response to new events).

[0054] Finally, it should be emphasized that rules-based approaches to these problems are simply not feasible. For example, when humans manually recognize that certain types of anomalies are common to a particular scenario (e.g., conflicts between the quantity of items in a PO and an associated invoice), programmers theoretically could implement a rule to identify and address such anomalies. But, as a practical matter, there could exist thousands or even millions of different types and variations of such anomalies. Expecting humans to pre-identify such anomalies and determine how each should be addressed would be an exercise in futility.

[0055] Current rules-based approaches handle only a small fraction of actual anomalies in any non-trivial scenario, leaving the rest to human judgment and intervention. There is thus a need for an automated system that detects and addresses such anomalies without requiring that humans derive rules in advance for implementation by programmers.

[0056] To address the above problems, and replace much of the existing overreliance on human judgment and intervention with automated means of deriving meaning from context, such an automated system would perform some or all of the following: [0057] 1 Process documents as events occur—e.g., as information is obtained or provided by a customer in the course of the transactions and related processes defined by the customer's particular use case scenario (whether in the realm of financial business transactions or otherwise); [0058] 2 Employ contextual semantic derivation (CSD) in the performance of document-processing and/or higher-level tasks designed to derive meaning from the current context of documents already processed (including identifying and addressing anomalies that constitute DDRs—i.e., data interrelationships within and across documents that are difficult to discern from the explicit information contained within documents); [0059] 3 Represent such derived relationships among data within and across documents (including DDRs) in one or more knowledge graphs, and iteratively update such knowledge graphs automatically on an ongoing basis; and [0060] 4 Respond to events (including receipt of new documents or other information, internally-generated and externally-generated natural language or other queries, passage of time, detection of an anomaly, anticipation of expected future events, hypotheses of potential events, etc.) based upon the current state of the knowledge graphs, which may involve employing CSD to perform or reinvoke tasks, and update knowledge graphs accordingly.

[0061] Such a system would employ a proactive holistic approach (rather than one that relies on predefined rules and isolated applications of AI-based technologies triggered by customers) to automate much of the work currently being performed by humans to derive meaning from context in the form of relationships among data within and across documents. Many such relationships, including DDRs, would be derived automatically, thereby limiting the need for human judgment and intervention.

[0062] Existing approaches have failed to represent the current state of such relationships in iteratively updated knowledge graphs, much less utilize that current state on an ongoing basis to perform document-processing and higher-level tasks over time. Doing so would greatly reduce the reliance on AP, AR and other human staff by simulating the inferences currently requiring intervention by such human personnel.



## SUMMARY

[0063] In accordance with the present invention, various embodiments of novel methods and architectures are disclosed herein. The present invention provides a novel solution to the problems described above in the form of an autonomous event-driven integrated system designed to run on a continuous basis to implement virtually any scenario in which relationships among data within and across documents are difficult to discern (without human intervention) from the explicit information contained in those documents.

[0064] As noted above, existing overreliance on human judgment and intervention has resulted from a failure to derive meaning from context in an automated fashion. Existing attempts to automate the processing of documents have been insufficient to detect, much less address or resolve, the anomalies in data interrelationships, particularly DDRs, that are prevalent in virtually any non-trivial use-case scenario.

[0065] Unlike existing solutions, the present invention integrates the automated performance of various tasks (including document-processing and higher-level tasks) that are common to any given scenario. Users of the present invention need not determine when and to what extent to invoke or reinvoke particular tasks. Moreover, the ability of the present invention to identify and address DDR-based and other anomalies is not dependent on predefined rules (which, in any event, could not feasibly be determined in advance).

[0066] The integrated system of the present invention runs autonomously on a continuous event-driven basis. As events occur (including among others the receipt of new documents, the detection of an anomaly, the absence of an expected document over time or even the anticipation of an expected or potential event), the present invention employs CSD to derive meaning from context. In one embodiment, this occurs in the processing of documents to identify data interrelationships (both within and across documents), including DDRs. These derived relationships are employed to update one or more “Knowledge Graphs,” which are in turn used over time for subsequent processing of documents and other events.

[0067] In other embodiments, the present invention also employs CSD to perform higher-level tasks, such as decisioning, interrogation, analytics, narrative generation, anomaly detection and resolution, etc. The process is somewhat similar to that of document processing, in that meaning is derived from context in the form of data interrelationships, including DDRs, already present in one or more Knowledge Graphs as well as in current data being processed (whether in the form of documents, queries, predefined workflows, etc.).

[0068] This proactive holistic approach automates much of the work currently performed by humans. Automatically deriving meaning from context reduces the need for human judgment and intervention.

[0069] In one embodiment, a “Scenario Controller” manages the overall logistics of the continuous event-driven process, including the scenario-specific aspects of the process, such as conditional staff approvals and workflow, particular task definitions, document types, event types, etc. The Scenario Controller also manages the invocation of events and the various tasks performed in the course of responding to events to implement the scenario.

[0070] While some of these tasks may still require some human judgment and intervention, the Scenario Controller manages specialized task-specific agents (“TSAgents”) designed to employ CSD to automate the performance of as much of the processing of documents as possible. In other embodiments, the TSAgents also implement higher-level tasks, including detecting and addressing anomalies, making decisions to authorize payments, ensuring the performance of approvals and other interim actions and responding to analytics requests, natural language user queries, etc.,

[0071] As noted above, tasks are performed on an ongoing basis in response to events. By employing CSD to derive data interrelationships (including DDRs) and iteratively update one or more Knowledge Graphs, the present invention greatly reduces the reliance on human staff by simulating the inferences currently requiring human judgment and intervention.

[0072] In particular, as will be explained in greater detail below, the iteratively updated Knowledge Graphs contain the context necessary to enable the present invention to detect and address anomalies that would otherwise go undetected, or would require significant human judgment and intervention. Addressing such anomalies may require that certain tasks be reinvoked over time, whether due to the presence of new information (including the mere passage of time) or simply the contextual knowledge of the existence of such anomalies.

[0073] As will be illustrated in greater detail below, multiple iterations may be required to adequately address certain anomalies. While not every anomaly can be resolved without the necessity of human judgment and intervention, the present invention significantly reduces the need for such human assistance.

[0074] As noted above, derived relationships among data both within and across documents often include DDRs, in part due to the existence of many-to-many relationships. A particular field or other document component may be related to a field in another document which may or may not be part of the same transaction. For example, a supplier name on a PO may match the supplier name on an invoice, but for a completely different transaction.

[0075] The ability of the present invention to segment individual documents (e.g., multiple POs or other document types on a single page, as well as a single PO that spans multiple pages), as well as classify multiple documents (and their components) and match them to particular transactions, is one of the many characteristics of the present invention that distinguishes it from existing systems that too often rely on human judgment and intervention in these scenarios.

[0076] As the TSAgents perform these tasks on an iterative basis over time, these derived relationships (including DDRs) are reflected in one or more iteratively updated Knowledge Graphs. As a result, the Knowledge Graphs represent, at any given point in time, the underlying relationships from which a current narrative or story behind the transactions, documents and other related information previously processed by the system can be generated. The Scenario Controller determines when to invoke particular TSAgents to perform or reinvoke certain tasks, based in part upon the current state of the Knowledge Graphs, as well as the occurrence of one or more events.

[0077] In one embodiment, such events include (among others) receipt or absence of new documents and other information, passage of time, detection of an anomaly, anticipation of expected documents or other information, hypotheses of potential future conditions, updates to the Knowledge Graphs, etc.). In response to such events, the Scenario Controller triggers one or more TSAgents to perform or reinvoke certain tasks, which results in further updates to the Knowledge Graphs. This iterative process operates on a continuous basis over time, with the Knowledge Graphs frequently changing to reflect the current state of a vast array of data relationships (including both semantic and temporal ones).

[0078] It should be noted that the autonomous nature of the present invention results in the Scenario Controller triggering TSAgents in response to a wide variety of internally-generated as well as externally-generated events, including updates to the Knowledge Graphs themselves. In other words, the present invention is continuously learning and searching for new patterns based on Knowledge Graphs that are constantly being updated (e.g., due to the passage of time as well as the receipt of new documents or other information).

[0079] In one embodiment, the Scenario Controller not only anticipates future conditions (e.g., an expected invoice), but also hypothesizes potential future conditions (e.g., based on detection of a learned pattern). For example, the Scenario Controller may receive an invoice on the same day each month for the same amount, and routinely forward that invoice to a particular supervisor for approval. At some point, the Scenario Controller detects this pattern and hypothesizes a more efficient solution (from its learned experience)—an “auto-approval” that enables the supervisor to approve automatically in advance all future invoices received on the same day of the month for that same amount.

[0080] In this manner, the present invention proactively generates a suggestion to the supervisor for

approval of this streamlined process—just as a skilled employee might do. The autonomous nature of the present invention, operating on a continuous basis over time (with a constantly changing state of data relationships reflected in the Knowledge Graphs) makes this level of proactivity possible with minimal human intervention.

[0081] In one embodiment, a “Narrative Generator” is employed by the Scenario Controller to traverse the Knowledge Graphs to facilitate the determination of the “next step” in the workflow specific to a customer's scenario (e.g., taking an action relating to an upcoming decision, processing additional documents, re-processing documents to complete a particular task based on updated information in the Knowledge Graphs, traversing the Knowledge Graphs to respond to internally or externally generated natural language queries, etc.).

[0082] The Narrative Generator, in another embodiment, also generates internal hypothetical queries, such as determining whether there will be sufficient funds to pay all invoices at the end of the month, or whether transfers from other bank accounts will be necessary. Moreover, the Narrative Generator also responds to user-generated natural language queries that pose hypothetical questions. In one embodiment, the Narrative Generator “temporarily” modifies the Knowledge Graphs to analyze such hypothetical situations.

[0083] It should be noted that this process is also an iterative one in that tasks may well be triggered by updates to the Knowledge Graphs or by other events (e.g., receipt of new documents or other information, the passage of time, etc.). Individual TSAgents (e.g., performing capture, classification, verification and other document-processing tasks) may be triggered at any time as the Scenario Controller monitors the occurrence of events (including updates to the Knowledge Graphs) to determine, for example, whether recently obtained information (even a newly derived relationship among components of documents previously processed) warrants the current performance of a task involving further processing of documents, a workflow-specific action or decision, or a response to a natural language user query, among other tasks.

[0084] In one embodiment, documents are processed by the system in an asynchronous manner. For example, system users may provide a “document package” to the system for batch processing of documents, both structured and unstructured. A document package may contain one or more documents in virtually any format (handwritten or printed paper, emails, spreadsheets and database files, streaming audio, video or other media files, etc.).

[0085] Users may also provide individual documents (or document packages) to the system over time, for example as the information is obtained (such as an email or invoice recently received). Users need not separate individual documents, pages or even document types, as the system performs the task of capturing and classifying such documents (in one embodiment by a TSAgent designed for that particular task, including segmenting individual documents).

[0086] TSAgents employ CSD as necessary to capture and classify documents and document types, as well as identify individual document components, such as data fields at various levels of abstraction (as well as other document processing and higher-level tasks). In the course of performing such tasks, TSAgents derive meaning from the context of all documents previously processed in the form of relationships (among documents, entities and other document components) that are maintained in the iteratively updated set of Knowledge Graphs.

[0087] It should be noted that such relationships may not all be derived as each document is processed. Instead, the TSAgents derive certain relationships over time in response to the occurrence of events, including detecting the absence of documents or other information expected in the future. The present invention detects over time not only the receipt of new information, but also anomalous events, including the absence of documents and other information. It also anticipates the occurrence of expected events and hypothesizes the occurrence of potential events, and responds proactively by invoking one or more TSAgents.

[0088] For example, the system may identify an invoice, but not be able to match that invoice to any particular purchase order until subsequent information is obtained in the course of processing

future documents that provide sufficient context to enable the system to make such a determination. The absence of such expected information is reflected in the Knowledge Graphs.

[0089] As the scenario progresses, the Scenario Controller encounters new information that involves not only further processing of documents, but also decisions and component actions dictated by the particular scenario workflow. Moreover, users may request that the system perform various analytics on the current state of the scenario, as reflected in the Knowledge Graphs.

[0090] In one embodiment, users provide natural language queries to the system, thereby automating many of the tasks routinely performed by humans, such as AP and AR staff members. Unlike highly constrained database queries, these natural language interrogations are not merely translated into standard SQL and other database queries. Instead, as is illustrated below, these queries involve the system's traversal of the Knowledge Graphs, including the many relationships among data not easily discerned or feasibly stored in a traditional database.

[0091] The application of CSD by particular TSAgents derives meaning represented by such relationships stored in the Knowledge Graphs. For example, despite missing or contradictory information stored in the explicit text of particular documents (such as the payment amounts of particular transactions), TSAgents may derive the actual information from the context provided by other documents (such as summary reports) and respond to a user's natural language query without error or equivocation.

[0092] As noted above, the embodiments of the novel methods and systems of the present invention, while described herein with respect to financial transactions, are equally applicable to personal, business, educational, governmental and other processes, including those outside the financial realm. Moreover, the descriptions of paper and electronic documents and files herein encompass a definition of "documents" well beyond individual characters and words, including their locations on a page or in a file, the structure of data and other higher dimensions of documents. In short, documents include any metadata or attribute of information from which signal can be distinguished from noise to reveal meaning.

[0093] Finally, the present invention provides an enhanced manner of training neural networks and other machine-learning models beyond the traditional uses of numerous training samples. To facilitate real-time modified training of models, the present invention, rather than generating synthetic data, utilizes a "Validation Engine" (consisting of multiple trained models) on real-time data when confidence levels are insufficient to merit conclusions, and retrains its existing models based upon the output of the Validation Engine. In this manner, existing models are retrained in real time based upon actual customer data without the additional time and expense required to generate additional training samples or synthetic data.

[0094] Many additional examples of the automation of tasks currently left to human judgment and intervention will be apparent from the description of the various Figures below.

---

## Description

### BRIEF DESCRIPTION OF DRAWINGS

[0095] FIG. 1 is a system-level block diagram of one embodiment of key modules the system of the present invention;

[0096] FIG. 2 is a dynamic process-level state/flow diagram of one embodiment of the functionality performed by key modules of the present invention;

[0097] FIG. 3 is a dynamic flow diagram of one embodiment of the real-time validation engine and self-learning functionality performed by key modules of the present invention to retrain models from actual processed data;

[0098] FIG. 4 is a screenshot of one embodiment of an interactive session of the present invention in which an AP clerk submits a recent invoice to the present invention for processing.

## DETAILED DESCRIPTION

[0099] Embodiments of the present invention described below include novel architectures and methods for automating the derivation of meaning from the context “hidden” within and across documents to reveal a more complete picture of the underlying narrative of the transactions and related processes spawning such documents. As noted above, while these embodiments focus on scenarios involving financial business transactions, the underlying principles are equally applicable to personal, business, educational, governmental and other processes, including those outside the financial realm.

[0100] Turning to FIG. **1**, key components of one embodiment of the present invention are illustrated in system diagram **100**. While not illustrated explicitly in FIG. **1**, components of system **100** are implemented as software modules embodied in physical non-transitory computer-accessible storage media (i.e., memory) from which they are invoked for execution by one or more CPUs or other physical processing units on standard computer hardware, such as interconnected computer servers, desktops, laptops and other similar devices. Also not shown are other standard physical computer components (including memory, displays, I/O devices, communication and networking devices, etc.) and external networks (e.g., the Internet) with which system **100** communicates.

[0101] It should be noted that the functionality of the modules of system **100** can be implemented in combinations of software and hardware in accordance with standard engineering tradeoffs. Moreover, such functionality can be merged together, split apart and otherwise reallocated into subsets or supersets of the modules illustrated in system diagram **100**. Similarly, the tasks performed by one or more such modules can differ from those illustrated and described herein without departing from the spirit of the present invention.

[0102] In one embodiment, each user scenario (personal, business, financial, non-financial or otherwise) defines its own specific tasks as well as document types, relevant fields and other components, and events to which system **100** will respond. Moreover, as noted above, documents of each particular document type may include structured and/or unstructured information, text, audio, video and other media—i.e., virtually any medium, metadata or other attribute of information from which “signal” can be distinguished from “noise” to reveal meaning. In other embodiments, system **100** defines general scenario implementations which customers accept and/or customize to a desired extent.

[0103] As alluded to above, Scenario Controller **102** manages the workflow defined by the scenario. In one embodiment, this workflow includes various approvals, payments, verifications and other decisions and component actions performed by particular personnel as part of a process resulting in one or more desired outcomes.

[0104] As a general matter, Scenario Controller **102** also manages the overall logistics of the continuous event-driven process, including the identification and invocation of events and the triggering of various tasks performed in the course of responding to such events to implement the scenario. As will be discussed in greater detail below, Scenario Controller **102** identifies and monitors the occurrence of events and, in response, invokes specialized TSAgents that employ CSD to automate the performance of various document-processing and higher-level tasks.

[0105] Scenario Controller **102** determines when and to what extent to invoke particular TSAgents to perform these tasks based in part upon the current state of Knowledge Graph **125**, as well as the occurrence of one or more events. In one embodiment, Knowledge Graph **125** is implemented as a single entity representing relationships (including DDRs) derived by one or more TSAgents. In other embodiments, Knowledge Graph **125** is implemented as multiple distinct but related entities, each referred to herein collectively as Knowledge Graph **125**.

[0106] As noted above, Knowledge Graph **125** is frequently updated as various tasks are performed to reflect the current state of a vast array of data relationships (including both semantic and temporal ones) both within and across documents. As will be discussed in greater detail below,

these relationships, including those derived by TSAgents via the use of CSD, often represent DDRs (typically across multiple relationships) that reflect errors and other anomalies which are difficult to discern without human judgment and intervention. The current state of Knowledge Graph **125** enables Scenario Controller **102** and other modules of system **100** (including their TSAgents) to traverse the current state of Knowledge Graph **125** to determine how best to perform their tasks on a continuous basis over time.

[0107] In one embodiment, Knowledge Graph **125** updates are implemented via standard Knowledge Graph Language (KGL) change requests. In other embodiments, a host of other graphing and query languages alternatives are employed, such as Resource Description Frameworks (RDF), Web Ontology Language (OWL), SPARQL, Cypher, GraphWL, Gremlin, etc. The choice of centralized v. decentralized models, graph storage options, NLP and semantic enrichment alternatives, and various hybrid implementations of the above involve design and engineering tradeoffs that can be accommodated without departing from the spirit of the present invention.

[0108] In another embodiment, Knowledge Graph **125** employs semantic, temporal and activation weights respectively to facilitate interpretations of and responses to natural-language queries, actions accounting for the relevance of time and biases relating to recently activated nodes. Moreover, the use of “Retrieval Agents” and “Hypothesis” Agents, which employ the semantic, temporal and activation weights, facilitates responses to hypothetical as well as actual events (e.g., to assess the effect on monthly budgets of doubling current orders for a particular product). Here too, the choice of how to implement particular weighting schemes as well as Retrieval and Hypothesis agents is the result of design and engineering tradeoffs that can be accommodated without departing from the spirit of the present invention.

[0109] Scenario Controller **102** also communicates with “external” persons, entities, systems and other devices **197** via External Communicator **195**. In one embodiment, the Internet (not shown) is employed to facilitate this communication, enabling Scenario Controller **102** to communicate bidirectionally with users of system **100** and third-party systems (e.g., to obtain new documents, third-party data feeds and other information for processing, as well as to receive and respond to user queries and analytics requests, among other purposes).

[0110] Other components of system **100** illustrated in FIG. **1** relate to the particular tasks performed with respect to a given scenario. While many of these components are common to many different scenarios, certain tasks may be added, revised or eliminated without departing from the spirit of the present invention. In the illustrated embodiment, each of these tasks involves, at least in part, the performance of CSD to derive meaning in the form of relationships among data within and across documents.

[0111] In the course of facilitating transactions and other processes that result in the generation and/or processing of related documents, Scenario Controller **102** relies on various document-processing TSAgents, which in this embodiment are specific to the various tasks defined by the scenario. One such TSAgent is Capture and Classification Engine **110**, which performs data capture tasks, as well as document classification (e.g., among relevant data types) and identification of particular document components, such as various fields of data.

[0112] While some of the functionality performed by Capture and Classification Engine **110** is relatively straightforward, there are many instances in which information may be missing, incorrect and contradictory with respect to similar information found within and across documents. As noted above, handwritten errors (often compounded by OCR errors) as well as data entry errors, are quite common. Moreover, the lack of standard document templates often exacerbates this problem.

[0113] In this embodiment, Capture and Classification Engine **110** employs CSD in an effort to derive meaning from context, which may involve analysis of other documents in a document package, or analysis of future documents at a later time, to complete its task. By deriving relationships among data, both within and across documents, and storing derived relationships in

Knowledge Graph **125**, Capture and Classification Engine **110** lays the groundwork for analyzing DDRs and identifying and addressing DDR-related errors and other anomalies.

[0114] While it employs various AI-based techniques to perform its task, a significant aspect of Capture and Classification Engine **110** is its use of CSD to derive many-to-many and other relationships from the context available not only in the current document or documents it is processing at any given time, but also from future information that may “fill in the blanks” at a later time and enable it to complete its task.

[0115] For example, a capture error regarding a vendor name in an invoice may not be recognized initially, or may be captured incorrectly. But, during a subsequent iteration, Capture and Classification Engine **110** may be reinvoked with an updated Knowledge Graph **125** containing a matching PO with an accurately captured vendor name. The additional context provided by other fields (e.g., within the invoice, as well as within a matching PO or other documents) enables Capture and Classification Engine **110** to identify and correct the initial vendor name capture error in the invoice.

[0116] In one embodiment, the inability to capture a vendor name with sufficient confidence is stored in Knowledge Graph **125**, thus facilitating the subsequent correction. Even if an incorrect vendor name was captured with sufficient confidence and stored in Knowledge Graph **125** (e.g., due to a human handwritten or data entry error), Capture and Classification Engine **110** may still be able to correct the error due to the existence in Knowledge Graph **125** of the additional context provided by other fields within and across multiple previously processed documents.

[0117] In this manner, Capture and Classification Engine **110** iteratively performs both capture and classification functionality as it encounters each document, updating Knowledge Graph **125** accordingly with the relationships it is able to derive at any given point in time. As noted above, the absence of predefined document templates in many scenarios presents a particularly difficult classification problem—i.e., recognizing delineations among distinct documents. Here too, multiple iterations may be required to correctly segment documents for classification.

[0118] For example, consider a written page containing two handwritten invoices containing a significant number of errors in field values and layout. Such errors may prevent Capture and Classification Engine **110** from properly capturing and segmenting the page initially, concluding that only a single invoice is present on the page, or perhaps that there is insufficient confidence to make a determination, in either case updating Knowledge Graph **125** accordingly.

[0119] Upon subsequent iterations with an updated Knowledge Graph **125**, Capture and Classification Engine **110** has access to sufficient context to determine that two invoices are present on the page, and to identify corresponding fields and field values more accurately. For example, in the interim, Capture and Classification Engine **110** may have encountered two matching POs, enabling relevant fields in the two invoices to be captured more accurately and matched to their respective POs.

[0120] In a similar manner, Matching and Reconciliation Engine **120** performs the matching and reconciliation tasks alluded to previously. As noted above, the existence of numerous different data fields increases exponentially the number of potential mismatches. Whether or not capture and classification errors have been identified and/or corrected, mismatches among data fields (both within and across documents) may well remain.

[0121] For example, a payment remittance document may or may not clearly indicate how a specified payment should be allocated among one or more invoices. As noted above, many-to-many relationships are often present—e.g., in which a single payment is intended to be matched to multiple invoices, or a single invoice may be satisfied with multiple payments.

[0122] It should be noted that Matching and Reconciliation Engine **120** may not be able to derive all of these relationships initially. The amount specified on a payment remittance document may not correspond to the amount of any invoice (or invoices) previously encountered—a tentative “mismatch” that, in one embodiment, is reflected in Knowledge Graph **125**. Upon subsequent

iterations, an updated Knowledge Graph **125** may provide Matching and Reconciliation Engine **120** with sufficient context to conclude that the payment was intended to be allocated to one or more invoices, or to a portion of a single invoice.

[0123] If, for example, the amount specified on the payment remittance document equals the sum of the amounts on two relevant invoices for which the payment was intended, no further reconciliation may be necessary (except perhaps automatically notifying relevant human personnel). If, however, a mismatch remains, then Matching and Reconciliation Engine **120** eventually may have sufficient context to determine that the amount specified on the payment remittance document (or on one or both invoices) was in error, in which case a corrective action may be necessary.

[0124] In one embodiment, certain corrective actions are completely resolved automatically (such as correcting an inaccurate payment amount) while others are addressed by notifying relevant personnel (e.g., an AP clerk) automatically that a corrective payment or refund is warranted. Whether or not humans are ultimately involved in performing follow-on corrective actions, Matching and Reconciliation Engine **120** automatically identifies mismatches and reconciles them.

[0125] As noted above, reconciling mismatches often involves determining contextually whether there exist “good” or “bad” reasons or justifications for one or more mismatches. For example, if a company has a “good” reason for paying a supplier a lower amount than was specified on a matching invoice (e.g., due to the receipt of some defective units of a product), then reconciliation may involve a simple corrective action such as a subsequent notification of the reasons for the prior lower payment. But if the lower payment amount was due to a “bad” reason (e.g., the failure of the company to realize that the correct number of units arrived in two different shipments), then reconciliation may involve a subsequent payment by the company for the difference.

[0126] Apart from Capture and Classification Engine **110** and Matching and Reconciliation Engine **120**, additional document-processing TSAs include Verification and Compliance Engine **122** and Coding Engine **124**. As discussed above, Verification and Compliance Engine **122** employs CSD to derive meaning within and across documents to validate discrepancies at various levels of abstraction (e.g., confirming that the value of a Tax ID is correct, despite it being in the correct format, or that component subtotals in certain documents add up to the total specified in another document).

[0127] In this embodiment, it also performs compliance tasks, such as ensuring compliance with various legal and regulatory requirements. Here too, CSD may be necessary to derive meaning from the context provided by multiple documents. Absent sufficient information to determine compliance at any given time, Verification and Compliance Engine **122** updates Knowledge Graph **125**, if only to reflect the future information required to complete its task.

[0128] Similarly, Coding Engine **124** employs CSD to determine how to fill in (or correct) data fields relating to standard accounting codes or custom company-specific codes. Both Verification and Compliance Engine **122** and Coding Engine **124** may require further processing at a future time to complete their respective tasks. Both similarly update Knowledge Graph **125** to reflect the information currently obtained, as well as, in one embodiment, missing information that may require human intervention or simply additional processing at a future time.

[0129] As noted above, detecting incorrect codes may appear to be relatively simple on the surface. But when the format of a substantively incorrect code is accurate, detection of such “hidden” errors becomes more complex, as it often requires contextual clues within and across documents (including external data sources and devices **197**). Once again, subsequent iterations may be required before such context is unearthed and reflected in Knowledge Graph **125** in the form of new and modified relationships.

[0130] By employing CSD to derive relationships, and updating Knowledge Graph **125** accordingly, the above-described document processing TSAs eventually obtain sufficient context (e.g., during subsequent iterations) to detect and reconcile many mismatches. In short, they



derive meaning from context (including DDRs) to uncover the mystery behind one or more actual or potential errors or other anomalies, and respond by suggesting and/or implementing corrective actions automatically.

[0131] It should be emphasized that, in one embodiment, document-processing TSAgents (and others discussed herein) are capable of employing CSD to derive meaning from context by relying on machine-learning models trained to address a variety of situations common to a particular scenario. Such models are therefore capable of addressing a variety of issues (such as capture, classification, matching, reconciliation, verification, compliance and coding, among others) that they have not specifically encountered before.

[0132] For example, mismatches between the quantity of items in a PO and an invoice may be common to many scenarios. But distinguishing between an error in the invoice (e.g., a capture or data entry error) and an incorrect assumption that the invoice corresponded to the PO (e.g., due to the fact that the actual corresponding invoice has yet to be processed by system **100**) can be quite a difficult task, even for human clerks. As noted above, it is simply not feasible to devise rules in advance to address adequately these and a myriad of other related situations.

[0133] Yet, by training models on documents embodying similar situations, such models learn to make these difficult distinctions over time, even when presented with a specific situation never before encountered. Moreover, in the example above, even if Matching and Reconciliation Engine **120** initially made an incorrect assumption that the current invoice matches the PO (e.g., due to other similar field values between the PO and invoice), system **100** is capable of recognizing and addressing this mistake during subsequent iterations, in many cases due in large part to iteratively modified Knowledge Graph **125**.

[0134] In other words, once the corresponding invoice was processed, Matching and Reconciliation Engine **120** updates Knowledge Graph **125** to reflect the newly derived relationships (both within the current invoice and across other previously processed documents and their component fields). These relationships include new relationships between the PO and the actual corresponding invoice, as well as modified relationships, such as the severing of relationships between the PO and the previously processed (non-corresponding) invoice.

[0135] In other embodiments, discussed in greater detail below, certain “errors” and other anomalies may not be detected by any of the document-processing TSAgents. In such cases, they may well be detected by a higher-level Anomaly Detector **135** and subsequently addressed by Anomaly Resolver **145**.

[0136] In one embodiment, models are trained on sets of documents that reflect a particular type of anomaly. For example, “quantity mismatches” are a relatively common type of anomaly that occurs in a myriad of different ways. In simple cases, the quantity of items in a single PO should match the quantity of items reflected in a single invoice. In more complex cases, the cumulative quantity of items across multiple POs should match the quantity of items in a single invoice, or the quantity of items in a single PO should match the cumulative quantity of items across multiple invoices.

[0137] Anomalies may include various different data entry or capture errors in the quantities specified in any of these documents, as well as errors in other “fields” that result in a mis-correlation of documents to the same transaction. In any event, models are trained to learn to recognize patterns and detect various different “classes” of common anomaly types.

[0138] Moreover, because Knowledge Graph **125** is iteratively updated to reflect new and modified relationships among data across documents, Anomaly Detector **135** not only detects the presence of a particular type of anomaly (e.g., when presented with a specific set of documents that it has never previously encountered), it also identifies relevant metadata, such as the specific relationships involved.

[0139] For example, Anomaly Detector **135** may detect a quantity mismatch among documents relating to a particular transaction, and may also (e.g., by traversing relationships in current Knowledge Graph **125**) detect a specific mismatch between a many-to-one quantity relationship

between multiple POs and a single invoice, all relating to the transaction. It then updates Knowledge Graph **125** to reflect the existence of this anomaly specific to that many-to-one quantity relationship (and any other affected relationships) and notifies Scenario Controller **102** of this new event.

[0140] Note that, in this example, the anomaly has yet to be resolved. For example, the mismatch may be due to a data entry or capture error in one of the POs or in the invoice itself. In one embodiment, the resolution of that anomaly is left to Anomaly Resolver **145** during subsequent iterations of system **100**. In a manner similar to that of training Anomaly Detector **135**, Anomaly Resolver **145** is trained on samples of sets of documents reflecting common anomaly resolutions.

[0141] For example, one common resolution may involve a correction to the quantity field of the invoice. Training of Anomaly Resolver **145** may well include many sample document sets in which that invoice quantity field is corrected. Similarly, other common resolutions may involve a correction to the quantity field in one of the POs.

[0142] In one embodiment, Anomaly Resolver **145** directs Scenario Controller **102** to reinvoke relevant TSAgents, such as Capture and Classification Engine **110** or Matching and Reconciliation Engine **120** to determine if such changes are warranted based on recent updates to Knowledge Graph **125**. As a result, TSAgents may make appropriate corrections to relevant fields and reflect such corrections in updates to Knowledge Graph **125**. Anomaly Resolver **145** then detects (during a subsequent iteration) that the anomaly has been resolved in a particular manner (e.g., by correcting the quantity field of the invoice), which it reflects in a further update to Knowledge Graph **125** for use by system **100** (which may, for example, notify users that this particular anomaly that was previously detected had been corrected).

[0143] In other embodiments, Anomaly Resolver **145** engages in hypothetical corrections (e.g., to different fields of the POs and invoice) before determining which correction (or combination of corrections) is warranted—i.e., which resolution of the anomaly is more likely. This process involves a similar process of reinvoking TSAgents, though with Knowledge Graph **125** reflecting the hypothetical nature of the relationship modifications.

[0144] In other situations, reinvocation of TSAgents may be unnecessary. For example, an anomaly may be resolved after a subsequent document is provided to system **100**. Or it may be resolved by notifying human staff to contact a vendor for a correction (e.g., as opposed to correcting a capture error automatically). In any event, as will be illustrated in various scenarios discussed in greater detail below, Anomaly Detector **135** and Anomaly Resolver **145** work together to detect and resolve anomalies on a continuous basis over the course of one or more iterations of system **100**.

[0145] At various points in time, Scenario Controller **102** employs Narrative Generator **150** to traverse the current state of Knowledge Graph **125** to perform particular tasks. In one embodiment, at the end of each iteration, Narrative Generator **150** traverses Knowledge Graph **125** to generate a summary or “narrative story” of the state of the relevant transactions at that point in time. This narrative is iteratively regenerated over time as more and more information is processed by system **100**.

[0146] In one embodiment, Narrative Generator **150** employs a large language model (LLM) to implement its narrative generation capabilities. In other embodiments, standard LLMs are enhanced with the ability to traverse Knowledge Graph **125** and utilize the derived relationships (including DDRs) as a substitute for “prompts” from which it generates its summary narrative.

[0147] After a few documents have been processed, the narrative may reflect only a few transactions—e.g., one PO to a first supplier on a particular date and time, followed by two other POs to a second supplier. Over time, as more documents are processed, the narrative expands to reflect the additional documents and other information system **100** has processed. This may include a summary of multiple transactions and the associated documents and entities involved, as well as items sent or received at particular dates and times.

[0148] Given the knowledge of the scenario's workflow by Scenario Controller **102**, the narrative

may also include decisions and component actions implemented by various personnel. Payments may have been received and statements or invoices may have been sent, along with interim authorizations by specified parties.

[0149] For example, the workflow may require approval of an invoice for payment based on the satisfaction of certain conditions reflected in recently updated Knowledge Graph **125** (such as receipt of the invoice and receipt of each item ordered on the relevant POs). Prior to those conditions being satisfied, the narrative may simply indicate that authorization for payment is awaiting completion of those specific conditions.

[0150] Once Scenario Controller **102** determines that the conditions are satisfied (based on the state of the current narrative reflected in Knowledge Graph **125**, as updated by Narrative Generator **150**), Scenario Controller **102** generates an email to the relevant personnel requesting approval. In other embodiments, the specified workflow may permit the generation of an automated approval.

[0151] In one embodiment, upon determining that relevant workflow conditions are satisfied, Scenario Controller **102** invokes Decisioning Engine **175** to make “ultimate” decisions (e.g., implementing an approved invoice payment, approving a loan, etc.) as well as various interim actions along the way (e.g., interim approvals, notifications, etc.), as specified in the particular scenario's workflow.

[0152] In some embodiments, Decisioning Engine **175** also employs CSD to derive relationships (including DDRs) from the current state of Knowledge Graph **125**. For example, a decision or interim action may depend upon making inferences not clearly determined by the scenario's workflow. Should an invoice be paid when some of the items were defective, or delivered late? Should a loan be approved despite the borrower's having made a few late payments in the past?

[0153] These subjective decisions may necessitate human intervention or may be addressed by automated suggestions from Decisioning Engine **175** based upon its learned “experience” of handling similar situations during its training. While some of these inferences may qualify as anomalies detected by Anomaly Detector **135**, others may be made by Decisioning Engine **175** (or perhaps both modules). Because the outcome is not strictly rules-based, different models may result in different outcomes based in large part upon their training.

[0154] In addition to the modules of system **100** that handle the iterative processing of documents and other information (including each scenarios specified workflow), other modules respond to events generated by direct user interaction. For example, in one embodiment, Analytics Engine **180** processes natural language analytics-oriented requests based upon the current state of Knowledge Graph **125**.

[0155] Because users are not simply querying a database, far fewer restrictions exist on the nature and format of users requests. In one embodiment, LLMs enable Analytics Engine **180** to process natural language requests and answer unstructured free-form queries, such as calculating the average delivery time of various suppliers during particular months of the year for products matching specified conditions. Analytics Engine **180** derives such information based upon specified constraints included in users' requests by traversing the current state of Knowledge Graph **125**.

[0156] In other embodiments, Analytics Engine **180** also employs CSD to derive certain relationships (including DDRs) from Knowledge Graph **125**. For example, certain products may be delivered late for various reasons. If most such late deliveries were caused by the company ordering the products, should suppliers be “penalized” in the form of higher late-delivery percentages?

[0157] Some of these subjective judgment calls are, in some embodiments, made automatically by Analytics Engine **180**, while in others they are addressed via Narrative Generator **150**, which summarizes and explain multiple different late-delivery percentages accompanied by explanations of typical reasons for certain types of late deliveries. The training of Analytics Engine **180** dictates in large part the “behavior” of Analytics Engine **180**.

[0158] Similarly, users submit natural language queries to system **100** via Interrogation Engine **190**. LLMs facilitate the interpretation of such queries and the formulation of responses by Interrogation Engine **190**. Moreover, standard LLMs are enhanced to enable Interrogation Engine **190** to traverse Knowledge Graph **125** and utilize DDRs and other derived relationships to analyze queries and facilitate intelligent responses.

[0159] It should be noted that users of system **100** may include a myriad of affiliated individuals, companies and others. Users might be executives of a company, clerks in AR or AP departments, other employees in sales, marketing or other departments, well as third party affiliated companies and individuals.

[0160] For example, an AP clerk may ask whether the company has sufficient funds in a particular bank account to pay invoices due at the end of the month, or whether a transfer of funds from another account is warranted. While such a request might appear quite simple on the surface, traversal of Knowledge Graph **125** by Interrogation Engine **190** might reveal significant factors, such as the fact that certain invoices have yet to be approved or that others should have been received by now. Additional actions may therefore be necessary.

[0161] Factors such as these may have taken the AP clerk a relatively long time to discern from the company's internal databases. Instead, system **100** (in particular Interrogation Engine **190**) provides a virtually instantaneous response, along with additional information and/or proactive suggestions for further actions. Moreover, Interrogation Engine **190** provides the opportunity for interactive conversations, including follow-up questions to clarify certain details and unearth additional relevant factors (i.e., stemming from DDRs and other relationships contained within Knowledge Graph **125**).

[0162] Moreover, in one embodiment, Interrogation Engine **190** proactively generates conversational queries to users (e.g., asking whether an item should be included in a system-generated report, or whether an outstanding invoice should be escalated for prompt payment), as well as internally-generated system queries (e.g., a request for the balance of a bank account, which can be implemented without human interaction). In both cases, Interrogation Engine **190** obtains additional information to be processed by system **100** (i.e., as an alternative source of information as compared to processing new documents). Details of these and other user interactions with system **100** are discussed in greater detail below in the context of specific sample scenarios.

[0163] In other embodiments, Interrogation Engine **190** works together with Analytics Engine **180**, Decisioning Engine **175** and other modules of system **100** to enable interactive conversations with users regarding general queries as well as those relating specifically to particular analytics requests, decisions and other aspects of the current scenario. In this manner, users interact with the virtual equivalent of an extremely knowledgeable and efficient human clerk.

[0164] Finally, model trainer **107** is employed by system **100** to provide an automated “self-learning” mechanism that modifies trained models in real time from live document data (not synthetic data) processed by system **100**. It is important to distinguish this form of training from one which employs automatically generated “synthetic” training samples.

[0165] In one embodiment, when a “primary” trained model (processing actual live data) yields insufficient confidence levels to merit a result, model trainer **107** invokes multiple other models to process the same actual live data. It then employs a “voting” mechanism to determine the “winning” result among those generated by the other models—e.g., by selecting the result that garnered the highest confidence levels.

[0166] While different embodiments employ different voting algorithms to select the winner (e.g., majority vote, highest actual, average or median confidence level, etc.), the selected result will be utilized by system **100** to complete its current process (as if the original confidence level generated by the primary model was sufficient) and update Knowledge Graph **125**. In other embodiments, model trainer **107** is employed not only with respect to document-processing tasks performed by TSAgents, but also with respect to higher-level tasks (decisioning, analytics, interrogation, etc.)

performed by TSAgents and other modules of system **100**.

[0167] In addition to performing this voting functionality, model trainer **107** implements real-time self-learning by automatically retraining the primary model to reflect the ultimately selected result. As will be explained in greater detail with respect to FIG. **3** below, model trainer **107**, in one embodiment, generates a template (filled with data reflecting one or more selected “winning” results) from which it retrains the primary model. It should be emphasized that this template mirrors the actual live data that was processed (e.g., by a TSAgent), as opposed to artificially-generated synthetic training samples.

[0168] Turning to FIG. **2**, diagram **200** illustrates one embodiment of a dynamic view of the functionality performed by the key modules of system **200** described above. In particular, process-level state/flow diagram **200** illustrates how Scenario Controller **202** responds to events on a continuous iterative basis over time, automatically invoking (or reinvoking) TSAgents and other modules to perform various tasks at the appropriate time.

[0169] As will become apparent below, Scenario Controller **202** determines which tasks are appropriate at any given time based not only on the scenario-specific workflow, but on the current state of Knowledge Graph **225**, which reflects the various relationships among data (within and across documents and transactions) that system **200** has derived thus far.

[0170] For example, depending on the event(s) being processed, Scenario Controller **202** will invoke or reinvoke particular TSAgents to process current documents and other data, detect and/or resolve anomalies and perform other higher-level functionality described in greater detail below. Upon performing these various tasks, system **200** updates Knowledge Graph **225** to provide the various modules of system **200** with current information during the remainder of the present iteration, as well as during subsequent iterations.

[0171] Before system **200** “goes live” (see Start **201**), Model Trainer **207** trains the various models (whether implemented as neural networks or other forms of supervised machine learning) in a manner well known in the art. Model Trainer **207** performs standard supervised training with various different types of training samples, as well as subsequent real-time “self-learning” retraining **208** (while system **200** is live), as alluded to above and described in greater detail below with respect to FIG. **3**.

[0172] Once models are sufficiently trained **209**, Scenario Controller **202** continuously processes Events **205** on an iterative basis over time. As noted above, in one embodiment, various types of Events **205** include (among others) receipt or absence of new documents or other information, internally-generated and externally-generated natural language or other queries, passage of time, detection or resolution of anomalies, anticipation of expected future documents or other information, hypotheses of potential future conditions, updates to Knowledge Graphs, etc.

[0173] Many Events **205** arrive via External Communicator **295**, which, in one embodiment, facilitates two-way communication of data **297** with the outside world (users, third-party systems, etc.). For example, such data **297** will often be in the form of new documents provided by users of system **200** for processing, initially by Scenario Controller **202**. As noted above, documents are far more expansive than the characters and words found in typical paper or electronic documents or files. They include not only the locations of characters and words on a page and the structure of such data (e.g., in a database or spreadsheet file), but virtually any other medium, metadata or other attribute of information from which “signal” can be distinguished from “noise” to reveal meaning.

[0174] Upon detecting a New Document event **211**, Scenario Controller **202** invokes one or more document-processing TSAgents **210** to process that document. Such processing includes invocation of various such TSAgents **210** discussed above with respect to FIG. **1**, including Capture and Classification Engine **110**, Matching and Reconciliation Engine **120**, Verification and Compliance Engine **122**, Coding Engine **124**, and others in other embodiments.

[0175] In one embodiment, the operation of system **200** is an asynchronous event-driven process. For example, while Capture and Classification Engine **110** is processing the new document (in

response to New Document event **211**), another Event **205** might arrive, such as a User Query event **291**. In that case, Scenario Controller **202** invokes Interrogation Engine **290** to interpret and address (e.g., respond to) the user query while Capture and Classification Engine **110** is simultaneously processing the new document in parallel.

[0176] In this manner, system **200** responds to multiple events asynchronously over time, achieving significant improvements in efficiency. When system **200** detects interdependencies which prevent certain functionality from being performed simultaneously, it delays the processing of functionality until other dependent functionality is completed. For example, if Interrogation Engine **290** could not respond to a user query until a relevant document currently being processed by Capture and Classification Engine **110** had been classified, then the operation of Interrogation Engine **290** would be suspended until such classification was completed, even if that did not occur until a subsequent iteration of system **200**.

[0177] It should therefore be noted that, in one embodiment, the processing of even a single document by TSAgents **210** might require one or more iterations of system **200** to complete. For example, more information may be required to resolve different aspects of processing a new document (capture, classification, matching, etc.). Perhaps the value of a particular field of a document cannot be captured with sufficient confidence. In that case, once the processing of the new document has been otherwise completed, the TSAgents **210** will update Knowledge Graph **225** with the current results of that processing, including the uncertain nature of that particular field.

[0178] Moreover, subsequent documents or other information may require a reinvocation of one or more TSAgents **210** over time that may modify the prior results of processing that document. For example, upon processing a new document, Capture and Classification Engine **110** may determine that it actually contains two documents (e.g., two invoices).

[0179] In one embodiment, even after classifying the two invoices and capturing each of their component field values, it simply updates Knowledge Graph **225** and completes the current iteration. In other embodiments, additional TSAgents are invoked during the current iteration (e.g., Matching and Reconciliation Engine **120**), which may be unable to match either invoice to any corresponding PO(s) because such documents have yet to be processed by system **200**. In any event, during any given iteration, once the TSAgents **210** have been invoked or reinvoked by Scenario Controller **202** and completed their processing, each will update Knowledge Graph **225** with newly derived or modified relationships or other changes.

[0180] In this manner, the order in which documents are presented to system **200** does not materially impact the nature of their processing. Even if a document cannot be “completely” processed at a given point in time due to a dependency on subsequent documents or other information being provided in the future, system **200** has the capability of reinvoking relevant TSAgents **210** to resolve such dependencies when it later obtains such information.

[0181] In one embodiment, system **200** also notifies relevant personnel of certain dependencies and, in some cases, requests specific information in an effort to resolve them. Such information may be provided by users of system **200** in the form of new documents or mere responses to such requests (via text, audio or other medium) which themselves effectively constitute “new documents.”

[0182] As alluded to above, in addition to New Document events **211**, Scenario Controller **202** also detects and processes other Events **205**. For example, upon receiving a query from a user of system **200** (e.g., via External Communicator **295**), Scenario Controller **202** detects the User Query event **291** and invokes Interrogation Engine **290** to interpret and address that user query.

[0183] As noted above, in one embodiment, Interrogation Engine **290** employs trained LLM models to interpret user queries, with the additional capability of traversing the current state of Knowledge Graph **225** and interpreting them in the context of various relationships, including DDRs. Moreover, it also utilizes such context to determine how best to address user queries.

[0184] For example, once appropriately interpreted, a user query might directly or indirectly

request not only the retrieval of certain information (such as invoices matching specified criteria), but also the performance of certain actions, such as correcting a specific data field value, reconciling data across multiple documents, etc. In such cases, Interrogation Engine **290** formulates responses to the user query and updates Knowledge Graph **225** accordingly. As a result, Scenario Controller **202** may invoke or reinvoke other document-processing TSAs **210** (or other modules) during the current or future iterations of system **200**.

[0185] As noted above, in one embodiment, user queries involve two-way interactive conversations between system **200** and its users. Such conversations often occur across multiple iterations of system **200**, and result in the performance of a myriad of different actions, which Scenario Controller **202** implements by invoking or reinvoking document-processing TSAs **210** as well as other higher-level modules of system **200**.

[0186] As system **200** processes documents and performs various other actions over time, Scenario Controller **202** utilizes the current state of Knowledge Graph **225** to invoke (as well as detect) certain Events **205**. For example, in one embodiment, Scenario Controller **202** integrates the scenario-specific workflow into Knowledge Graph **225**. In other embodiments, this workflow is maintained separately.

[0187] In either case, Scenario Controller **202** utilizes this current “state” of system **200** to determine when the scenario-specific workflow dictates that particular decisions are warranted—i.e., that specified interim actions have been completed and other conditions have been satisfied. For example, the scenario-specific workflow may require the approval of particular personnel before an invoice can be paid or a car loan can be approved.

[0188] Once all such conditions have been satisfied, Scenario Controller **202** generates a Decision event **276** and invokes Decisioning Engine **275**. At that point, in one embodiment, Decisioning Engine **275** implements the decision by causing specified actions to occur in accordance with the scenario-specific workflow and the current state of Knowledge Graph **225**.

[0189] For example, in a relatively straightforward case, it may generate a Miscellaneous event **285** (and update Knowledge Graph **225** accordingly) that will cause Scenario Controller **202**, on a subsequent iteration, to notify specified personnel that a car loan had been approved, and perhaps even generate the approval notice to be sent to the applicant. In other embodiments, certain tasks are implemented by human staff, while others are performed automatically in response to system-generated Events **205**, by various other modules of system **200**.

[0190] As noted above, in some embodiments, Decisioning Engine **275** employs CSD to derive relationships (including DDRs) from the current state of Knowledge Graph **225**, which may cause certain actions to be taken in connection with the decision. For example, even after approving a car loan, system **200** may initiate a subsequent stage of the scenario-specific workflow, in which loan documents must be signed, automatic loan payments must be set up with the applicant's bank, etc. Moreover, as noted above, certain subjective decisions may be delegated to human staff, or may be addressed via automated suggestions initiated by Decisioning Engine **175** based upon its learned experience of handling similar situations during its training.

[0191] Scenario Controller **202** also initiates Analytics events **281** in response to analytics requests from users of system **200**. Although analytics requests from users are, in essence, a subset of user queries, Scenario Controller **202** invokes Analytics Engine **280**, a module that is specifically trained to implement analytics-related requests.

[0192] For example, in addition to interpreting natural-language prompts and traversing Knowledge Graph **225** (e.g., via modified LLM-based models), Analytics Engine **280** is trained to perform various analytics-based functions, such as applying a myriad of statistical functions with specified constraints to data relationships derived from Knowledge Graph **225**. In one embodiment, employing CSD enables Analytics Engine **280** to make inferences beyond those typically discernible from traditional databases.

[0193] For example, as noted above, data relating to various late deliveries may be available in

Knowledge Graph **225**. But whether particular late deliveries were justified requires a much more complex analysis, including inferences enabled by the training of Analytics Engine **280** with respect to various similar situations. In another example, Analytics Engine **280** could discern that a particular supplier tends to make mistakes in item quantities regarding particular products, and suggest that another supplier be employed for future orders of those products.

[0194] In essence, Analytics Engine **280** “behaves” (based on its training) in a manner that enables it to implement a myriad of analytics-based user requests specifically tailored to the data of which it is aware (embodied in the current state of Knowledge Graph **225**). Moreover, it provides natural-language responses explaining the results of such requests, included detailed factual support data (names, dates, entities and various other relevant metadata) that system **200** has processed thus far.

[0195] Note that all of this functionality is performed asynchronously on an event-driven basis by system **200** while it is simultaneously handling the processing of documents and implementing various other higher-level functions. As a result, users experience an integrated system **200** that employs different trained models as needed to perform such functions.

[0196] In other embodiments, various other Miscellaneous events **285** are initiated by Scenario Controller **202**, and handled in a similar manner by other general-purpose or specifically trained models. To the extent the handling of such Miscellaneous events **285** results in the generation of new or modified relationships, such changes are reflected in updates to Knowledge Graph **225** for use during subsequent iterations of system **200**.

[0197] As described above, a particularly unusual and effective function of system **200** involves detecting and addressing (and in some cases resolving) anomalies. In one embodiment, during each iteration, after completing the specific event-driven document-processing or other higher-level function in response to one or more Events **205**, and updating Knowledge Graph **225** (reflected by a Knowledge Graph Updated event **226**), system **200** invokes Anomaly Detector **235** to traverse the current Knowledge Graph **225** in search of anomalies.

[0198] In one embodiment, when an anomaly is detected by Anomaly Detector **235**, it updates Knowledge Graph **225** with the details and other relevant metadata relating to the anomaly. In other embodiments, it provides such metadata directly to Scenario Controller **202**. In either case, on the next iteration of system **200**, Scenario Controller **202** issues an Anomaly event **246** and invokes Anomaly Resolver **245** to address the anomaly.

[0199] In the embodiment illustrated in FIG. 2, Anomaly Detector **235** is employed toward the end of each iteration of system **200** (when Knowledge Graph **225** has been updated and issues a Knowledge Graph Updated event **226**). This enables system **200** to detect anomalies based on all currently known data, with the resolution of any detected anomalies occurring during subsequent iterations of system **200**.

[0200] As noted above, Anomaly Detector **235** includes one or more models trained on sets of documents reflecting various different types of anomalies. For example, in one embodiment, such anomaly types include pricing anomalies (e.g., pricing errors, overpricing, unauthorized discounts, etc.), quantity anomalies (e.g., unauthorized orders based on item quantities, mismatches between POs and invoices, etc.), vendor fraud, contractual deviations, discrepancies in invoicing and payments, approval discrepancies, violations of laws or internal company rules, procurement delays (missed deadlines, poor supplier performance, etc.), unexpected changes in market conditions and supply chains, etc. In other embodiments, various other anomaly types are included in the training of Anomaly Detector **235** models.

[0201] As noted above, such training includes scenarios in which anomalies include DDRs both within and across documents. By analyzing such complex inter-document relationships reflected in Knowledge Graph **225**, Anomaly Detector **235** detects discrepancies across documents that It has never encountered before—i.e., based on its training on similar discrepancy scenarios, which enables it to learn to recognize similar patterns within various classes of common anomaly types.

[0202] Moreover, by traversing relationships reflected within Knowledge Graph **225**, Anomaly



Detector **235** not only detects the presence of a particular type of anomaly, but also identifies relevant metadata regarding specific relationships (e.g., many-to-many relationships between an invoice and multiple POs, correlating to a mismatch among quantity items). It then updates Knowledge Graph **225** to reflect the anomaly and such relevant metadata (or, in other embodiments, provides such information directly to Scenario Controller **202**) for use in resolving detected anomalies during subsequent iterations of system **200**.

[0203] During a subsequent iteration of system **200**, Scenario Controller **202** issues an Anomaly event **246** and invokes Anomaly Resolver **245** to address the anomaly (based on its training with respect to common resolutions of various different types of anomalies). As noted above, Anomaly Resolver **245** resolves certain anomalies simply and automatically without human intervention (e.g., correcting a faulty error in a data field, whether due to a capture or data entry error), while other anomalies require a more complex resolution.

[0204] For example, as noted above, certain anomalies require the issuance of another Event **205** relating to the anomaly, such as directing Scenario Controller **202** to reinvoke relevant TSAs (such as Capture and Classification Engine **110** or Matching and Reconciliation Engine **120**) to determine if such corrections or other changes are warranted based on recent updates to Knowledge Graph **225**. In other cases, Anomaly Resolver **245** initiates hypothetical resolutions to determine which resolution is most warranted.

[0205] In any event, upon resolving actual anomalies, Anomaly Resolver **245** directs Scenario Controller **202** to notify relevant users of system **200** (in accordance with the scenario-specific workflow) how the anomaly was resolved. In still other cases, Anomaly Resolver **245** suggests one or more potential courses of action to relevant users of system **200**, and leaves it to them to further implement the resolution of such anomalies.

[0206] As noted above, Anomaly Detector **235** and Anomaly Resolver **245** effectively work together to detect and resolve anomalies on a continuous basis over the course of one or more iterations of system **200**. Document-processing TSAs **210** and other modules of system **200** are also employed when warranted to facilitate both the detection and resolution of anomalies. Various specific scenarios illustrating some of the myriad of different circumstances in which anomalies are detected and resolved by system **200** are discussed in greater detail below.

[0207] In one embodiment, prior to completion of each iteration of system **200**, Narrative Generator **250** is invoked following the completion of Anomaly Detector **235**, whether or not an anomaly has been detected. In the vast majority of iterations, no anomaly will have been detected. Nevertheless, Narrative Generator **250** is employed primarily to summarize in a natural language format (for use both internally by system **200** and externally by relevant users) the narrative story of the state of the relevant transactions processed by system **200** up to that point in time.

[0208] It should be noted that this narrative story will be refined (i.e., iteratively regenerated) continuously over time as system **200** processes additional information. Ultimately, it will include a summary of multiple transactions, including relevant aspects of their associated documents and entities, as well as items sent or received at particular dates and times, payments made, decisions and component actions performed, interim authorizations, etc.

[0209] As noted above, summaries may reflect that certain decisions or component actions are awaiting authorization or satisfaction of other specified conditions, or that expected documents have yet to be processed by system **200**. In one embodiment, configurations of system **200** will determine the level of detail incorporated in such narrative summaries.

[0210] The narrative story generated by Narrative Generator **250** at the end of each iteration of system **200** is employed by Scenario Controller **202** in many instances to provide updates, responses and other feedback to users of system **200**. In other instances, Scenario Controller **202** utilizes the narrative story (in addition to the current state of Knowledge Graph **202** and the scenario-specific workflow) for the purpose of determining which Events **205** to initiate and which actions to perform during its current iteration.

[0211] The operation of Narrative Generator **250** as well as the above-described modules of system **200** will be better understood in the context of specific scenarios discussed in greater detail below. Before exploring such specific scenarios, the dynamic operation of the real-time validation engine and self-learning process illustrated in flowchart **300** of FIG. **3** will be discussed.

[0212] Flowchart **300** of FIG. **3** provides an illustration of this dynamic real-time validation and self-learning process. It should be noted that this process occurs whenever the primary trained models employed by system **200** (document-processing TSAgents **210** as well as other higher-level modules, including other TSAgents) at “inference time” generate results exhibiting confidence levels below predefined (or dynamically-generated) thresholds.

[0213] In other words, the key purpose of the process illustrated in flowchart **300** is twofold. One key objective is to complete the real-time functionality corresponding to the current inferences. In one embodiment, this functionality includes processing documents, interpreting and responding to user queries and analytics requests, initiating decisions or component actions dictated by the scenario-specific workflow, detecting and resolving anomalies, etc.

[0214] Another key objective of the process illustrated in flowchart **300** is to automatically enhance the training of such primary inference models in real time based upon actual data being processed (as opposed to synthetically generating training samples). In other words, this “self training” is performed in real time without materially delaying the operation of system **200**, unlike the significant delay that would result from standard retraining of models with synthetic or other training samples.

[0215] As a result, system **200** completes its current processing functionality, despite insufficient confidence levels in its primary models to merit generation of a conclusion. Moreover, it enhances the training of such primary models, based upon the actual data currently being processed (which resulted in insufficient confidence levels), thereby enabling such primary models to be retrained in real time for use during subsequent iterations of system **200**.

[0216] Starting at step **301** when the primary models are invoked at inference time (in step **302**), such models typically generate results that exceed predefined thresholds (in step **305**), enabling such primary models to complete their designated functions (in step **350**) and update Knowledge Graph **225** accordingly. In such cases, no real-time validation or self learning is necessary, and the process ends at step **360**.

[0217] For example, as illustrated in the context of a New Document event **211** in FIG. **2**, TSAgents **210** perform various document-processing functionality (capture, classification, matching, reconciliation, verification, compliance, coding, etc.) successfully before updating Knowledge Graph **225**). In other words, the capture and classification process performed by Capture and Classification Engine **110** yields conclusions regarding document types, component field values, etc.

[0218] But, in the event that any such conclusion (e.g., the classification of the current document into a document type, or the value of a particular captured component field) cannot be reached, due to an insufficient confidence level of the primary Capture and Classification Engine **110** model (i.e., a confidence level below a predefined threshold), then the process could not be completed in the normal fashion.

[0219] In other words, if Capture and Classification Engine **110** is unsure as to which type of document it is processing, or what is the captured amount of an invoice (because none of the available choices can be selected with sufficient confidence), then one alternative is to reflect that lack of certainty in Knowledge Graph **225** and continue to a subsequent iteration of system **200**. The same could be true of other document-processing TSAgents **210**.

[0220] But, in one embodiment, instead of waiting for additional documents or other information provided during subsequent iterations of system **200**, Validation Engine **310** of Model Trainer **207** is invoked in real time. In this embodiment, Validation Engine **310** employs a multi-model voting system that processes the same data provided, for example, to the primary model of Capture and

Classification Engine **110**, but using multiple other models trained to perform similar capture and classification functionality.

[0221] Validation Engine **310** selects a conclusion based, for example, upon the “majority vote” of these other models **325** (including LLM or other models **325a-325n**). In other embodiments, different algorithms are employed to determine the “winner” of the vote. In still other embodiments, the winner is determined by delegating the resolution to human personnel, or voting among less expensive alternative models **325**.

[0222] In any case, the result is the same. In other words, the winning result (e.g., document type is “invoice” or the value of the “amount” field of the invoice is \$100, etc.) is selected and included in updated Knowledge Graph **225** as if that result had been generated by the primary model of Capture and Classification Engine **110**. In other embodiments, Model Trainer **207** is employed not only with respect to document-processing tasks performed by TSAgents **210**, but also with respect to higher-level tasks (decisioning, analytics, interrogation, anomaly detection and resolution, etc.).

[0223] Having selected the winning result, that result is employed by the module (e.g., Capture and Classification Engine **110**) to complete its inference (at step **350**). Having achieved only one of the key objectives of this process, Model Retrainer **330** (of Model Trainer **207**) is also invoked to update the primary model in an effort to avoid similar problems during future iterations.

[0224] In other words, by utilizing the actual data that generated insufficient confidence levels as a basis for retraining this primary model, this retrained primary model is more likely to yield a conclusion with sufficient confidence when presented with data exhibiting similar issues during future iterations of system **200**.

[0225] In one embodiment, Model Retrainer **330** utilizes the actual data to generate a “training document template” in the form of a JSON file with automated annotations including a document type, labeled field values and identifiers reflecting the location of bounding boxes for each component field. It then employs this training document template to retrain the primary models **315** for use during subsequent iterations of system **200**.

[0226] In this manner, Model Trainer **207** achieves its dual objectives of completing the real-time functionality corresponding to current inferences of primary models (despite initial failures to reach conclusions with sufficient confidence) and automatically retraining such primary models based upon the actual data they just processed in an effort to avoid similar issues during subsequent iterations of system **200**. Moreover, it achieves these dual objectives substantially in real time without materially delaying the operation of system **200**.

[0227] Having described the static components and dynamic operations of key modules of system **200**, the following specific scenarios will be explored in order to better illustrate key advantages of the present invention. While no such descriptions can feasibly be exhaustive, they will serve to illustrate the many instances in which the present invention alleviates existing overreliance on human judgment and intervention by deriving meaning from context in an automated fashion.

#### SCENARIO 1 ANOMALY—Quantity Mismatch between POs and Invoices

[0228] This first scenario illustrates a common anomaly in which the quantities of items ordered from a supplier varies (or, in some cases, only appears to vary) from the quantity of items specified in one or more invoices. The “quantity” relationships between POs and invoices are but one example of DDRs in that they often occur in the context of many-to-many relationships—e.g., multiple POs reflected on a single invoice or a single PO reflected in multiple invoices.

[0229] As a result, it is often difficult (even for humans) to identify, much less reconcile, mismatches between the quantities of items ordered (across one or more POs) and those reflected in one or more invoices. Additional context (often reflected in other related documents) may be necessary to detect and resolve such anomalies.

[0230] In this scenario, a company (ACME Corp) orders personal computers (PCs) for its employees from a supplier (ACE Technologies). As shown in Table 1 below, two employees (Bob and Jane) request PCs from ACE for their business use. Per company policy, they typically order

PCs via requests to Alice, ACME's procurement officer, who issues POs to Steve, an ACE salesperson. ACE typically sends advanced shipping notices (ASNs) to Frank in ACME tech support before shipping products. ACE typically sends invoices for shipped products to Sam in ACME's AP department.

[0231] The actions performed by ACE and ACME employees in this scenario are summarized in Table 2 below. Note that these actions are described in chronological order in this example. Each documented action is submitted to system **200** for processing as it occurs in one embodiment. In other embodiments, packages of one or more documents (not necessarily in chronological order) are submitted to system **200** for processing at various points in time.

[0232] In any event, system **200** captures relevant dates and times as part of its overall process, enabling it to receive and process documents regardless of the order in which they were submitted. It should be emphasized that the capability of system **200** to determine when to reinvoke TSAgents **210** automatically is of particular significance with respect to its ability to process documents successfully when they are submitted "out of order" to system **200**.

[0233] The first action in this scenario involves Bob emailing Alice with a request to order a single PC. In one embodiment, this email is submitted promptly (and in some cases automatically) to system **200** for processing. Alice in turn generates a PO for that PC, which she sends to Steve at ACE. This PO is also submitted to system **200** for processing.

[0234] Note that system **200** captures the relevant data from this email and PO, as well as classifying these two documents accurately. While this aspect of the process appears to be relatively straightforward, it may require the use of CSD by Capture and Classification Engine **110** (as discussed above). For example, in one embodiment, the email and/or PO may have been manually printed and copied in a manner that accidentally created a mirror image of the actual document. Capture and Classification Engine **110** is trained to employ CSD to recognize common field locations and values, and thus properly capture and classify the email and PO (in an analogous manner to that of a human performing that task).

[0235] At this point in time, it should be noted that system **200** iteratively performs other document-processing tasks regarding these two documents, including (among other steps) updating Knowledge Graph **225** and invoking Anomaly Detector **235** to search for anomalies. As is usually the case, no anomalies are detected and system **200** proceeds to subsequent iterations looking for new Events **205**. Note that, as mentioned above, the mere passage of time is an Event **205** which will result at least in the updating of Knowledge Graph **225** to reflect that passage of time.

[0236] Subsequently, Jane desires to request a single PC. But, in this case, Jane violates ACME company policy and does not submit her request to Alice. Instead, she submits her request directly by phone to Steve at ACE (e.g., because she knows Steve well from frequent prior business interactions). As a result, her phone call is not documented and no PO is generated.

[0237] Moreover, even though Jane's order is fulfilled, Sam receives the invoice from ACE for both PCs before receiving any other relevant documents (such as a 2nd ASN, a receipt notice for one or both PCs, etc.). The invoice includes a reference to Jane's order from Steve, but does not include any other identifying information, such as a PO number.

[0238] Upon processing this invoice, system **200** employs Capture and Classification Engine **110** to classify the invoice correctly and capture relevant data fields. Matching and Reconciliation Engine **120** matches the first PO to this invoice, as well as the unit price of each PC and other data fields.

[0239] However, the total price listed on the invoice is double the PC unit price, and the quantity of 2 PCs does not match the single quantity of 1 PC listed on the PO. In one embodiment, Matching and Reconciliation Engine **120** identifies these "potential errors" and updates Knowledge Graph **225** accordingly, but does not "resolve the mismatch" as being caused by Jane's "out of policy" order. In other embodiments, Matching and Reconciliation Engine **120** effectively detects and resolves this anomaly itself.

[0240] In the current scenario, Anomaly Detector **235** identifies an anomaly upon traversing

updated Knowledge Graph **225**. In other words, based upon its training/experience, it recognizes the “mismatch” between the quantity of two PCs on the invoice and one PC on the only PO it has processed. It also recognizes that the total price on the invoice is double that of the PC unit price, and that a text field contains a reference to a second order from Jane to Steve.

[0241] In one embodiment, Anomaly Detector **235** updates Knowledge Graph **225** with the detection of the anomaly, along with potentially relevant metadata regarding the pricing info and reference to Jane's order from Steve. In subsequent iterations, Scenario Controller **202** generates Anomaly Event **246** and invokes Anomaly Resolver **245** to address the anomaly.

[0242] Anomaly Resolver **245** traverses updated Knowledge Graph **225** and identifies a likely resolution, based upon its training/experience, by detecting the common pattern of a missing PO number coupled with an undocumented order (i.e., the order from Jane to Steve referenced in a text field of the invoice). It resolves the anomaly by generating a recommendation to Sam that the invoice be paid.

[0243] In one embodiment, it updates Knowledge Graph **225** accordingly, causing Narrative Generator **250** to generate a natural-language recommendation to Sam, which Decisioning Engine **275** delivers to Sam after being invoked on a subsequent iteration by Scenario Controller **202**. At that point Sam may simply pay the invoice, or may require that additional conditions be met, which it conveys to system **200** via External Communicator **295**.

[0244] In another embodiment, Anomaly Resolver **245** also updates Knowledge Graph **225** to reflect the matching of Jane's order with the invoice. For example, in this embodiment, system **200** generates a “virtual PO” with a PO number that is also “included” on the invoice. As a result, Knowledge Graph **225** now contains the “missing context” as if Jane had followed ACME policy and issued an actual PO.

[0245] In other embodiments, various other actions are performed automatically in a more direct fashion. For example, modules of system **200** automatically generate notifications with suggestions to human staff for approval and/or implementation. Or anomalies may be detected, but not fully resolved until additional information is obtained, such as receipt of a “missing” PO, ASN or other document.

[0246] By notifying relevant users of system **200** promptly upon detecting an anomaly (i.e., a potential problem), such anomalies are resolved far more quickly than would otherwise be the case. These and other implementation choices are determined in accordance with standard logistical and engineering tradeoffs, as well as scenario-specific workflow specifications.

TABLE-US-00001 TABLE 1 Other ENTITIES Sales Procurement AP Tech Support Employees  
ACE Technologies - SUPPLIER Steve ACME Corp - COMPANY Alice Sam Frank Bob, Jane  
TABLE-US-00002 TABLE 2 HUMAN ACTIVITIES DOCUMENT COMMENTS Bob requests 1  
PC via Alice Email Alice issues PO to Steve at ACE PO Jane calls Steve at ACE \*\*\* N/A (Tel Call)  
No PO - “Out of Policy” ASN from Ace arrives to Frank re 1 PC ASN Matches PO Sam receives  
Invoice from Ace for 2 PCs Invoice Matches PO, PC Unit Price, etc. \*\*\* ALSO mentions Jane  
order via Steve

[0247] This scenario illustrates some of the myriad of different situations that are currently handled by human intervention, in which personnel spend significant time and money making the contextual inferences that system **200** performs automatically. While human intervention will likely never be eliminated entirely, system **200** performs much of this groundwork automatically, particularly when context is available (from information within and across documents) from which meaning can be derived.

[0248] Additional scenarios described below will further illustrate different aspects and advantages of system **200**. In addition to the wide variety of anomalies that system **200** can detect and resolve, it will become apparent that no rules-based pre-programmed system could feasibly “foresee” (employing humans as anomaly detectors and resolvers) the many different types and variations of anomalies whose “patterns” the trained models of system **200** can detect and address.

[0249] Moreover, as noted above, it will become apparent that the concepts illustrated by these scenarios with respect to financial business transactions apply equally to other types of personal, business, educational, governmental and other processes, including those outside the financial realm. What they share in common is the processing of tangible “documents” as broadly defined above (i.e., any metadata or attribute of information from which signal can be distinguished from noise to reveal meaning) that contain the context from which meaning can be derived (i.e., the relationships among data within and across such documents that are difficult to discern from the explicit information contained therein).

[0250] The continuous event-driven process of system **200** extracts such context from these documents on an iterative basis over time (by employing CSD within document-processing TSAgents **210**) and derives meaning from such context by detecting and resolving anomalies, as well as interpreting and responding to natural-language requests and directives while implementing scenario-specific workflows. The following additional scenarios illustrate the versatility of these concepts in various different circumstances.

#### SCENARIO 2 ANOMALY—Pricing Discrepancy between Contract and Invoice

[0251] This scenario involves the same personnel from ACME and ACE, but in the context of a pricing discrepancy. As described in Table 3 below, ACME and ACE enter into a contract including a section specifying that ACE quotes determine the pricing, and supercede any other contradictory document provisions.

[0252] Bob emails Steve at ACE requesting pricing for 1000 widgets. Steve emails a quote back to Bob indicating a unit price of \$10. Bob contacts Alice, who issues a PO to Steve for 1000 units at \$10 each. After some time passes, Frank receives an ASN from ACE indicating that 1000 units will be shipped at a price of \$11 each (due to a recent price increase).

[0253] In this embodiment, system **200** processes these documents as they are submitted chronologically. While processing the ASN, Matching and Reconciliation Engine **120** detects the pricing discrepancy between the \$11 unit price in the ASN and the \$10 unit price in the quote and updates Knowledge Graph **225** accordingly.

[0254] Anomaly Detector **235** confirms detection of this pricing anomaly, and additionally detects the potential relevance of the contract section relating to pricing. It updates Knowledge Graph **225** accordingly. In a subsequent iteration, Anomaly Resolver **245** determines that the quoted price of \$10 prevails, and addresses the anomaly by notifying Sam of this discrepancy. At this point, Sam elects to notify ACE of this error in an email.

[0255] Nevertheless, Sam eventually receives the invoice for \$11K (unit price of \$11), but does not immediately remember receiving the prior notification from system **200** of this pricing discrepancy, or his follow-up email to ACE. Fortunately, while processing the invoice, Matching and Reconciliation Engine **120** detects this same “mismatch” (unit pricing discrepancy) between the invoice and the PO, and updates Knowledge Graph **225** accordingly.

[0256] Once again, Anomaly Detector **235** confirms detection of this pricing anomaly as well as the relevance of the contract section relating to pricing. Note that the prior updating of Knowledge Graph **225** increases the confidence level of this result. Similarly, in a subsequent iteration, Anomaly Resolver **245** reconfirms that the quoted price of \$10 prevails (again with greater confidence).

[0257] Recognizing that system **200** had already notified Sam, and that Sam had emailed ACE, Anomaly Resolver **245** now resolves the anomaly by making a dual recommendation—i.e., (i) that Sam be notified again of the pricing discrepancy with an updated narrative, but also (ii) that the invoice be disputed due to the \$1K overpricing. Anomaly Resolver **245** also updates Knowledge Graph **225** accordingly.

[0258] As a result, Narrative Generator **250** is invoked to generate an updated narrative for Sam, which is delivered to Sam (during a subsequent iteration) by Scenario Controller **202** via External Communicator **295**. In addition, Scenario Controller **202** issues a Decision event **276** and invokes

Decisioning Engine **275**, which confirms the recommendation to Sam that the invoice be disputed (again employing Narrative Generator **250** to generate the updated narrative containing this recommendation). Sam then proceeds to dispute the invoice with ACE, resulting in a payment of \$10K per the contract.

[0259] In other embodiments, depending upon the scenario-specific workflow, additional actions could be automated, lessening human intervention even further. For example, system **200** could automatically generate and send notifications to ACE, along with payments of undisputed amounts (optionally with Sam's advance approval).

TABLE-US-00003 TABLE 3 HUMAN ACTIVITIES DOCUMENT COMMENTS ACE and ACME execute Contract CONTRACT Specifies that QUOTE determines PRICE Bob emails Steve requesting Pricing Email Seeking Pricing for 1000 Widgets Steve emails Bob with Quote Email/Quote \$10K for 1000 Widgets at \$10 each Alice issues PO to Steve at ACE PO ALSO \$10K for 1000 Widgets at \$10 each ASN from ACE arrives to Frank ASN \*\*\* 1000 Widgets at \$11 each (price increase) Sam email to ACE Email Error in ASN - Pricing should be \$10 Sam receives Invoice Invoice \$11K - 1000 Widgets at \$11 each

[0260] Note that, in this scenario, system **200** not only automates many of the inferences that typically would require human intervention (such as the detection and resolution of the pricing anomaly), but also provides levels of redundancy that increase the likelihood that such anomalies will be detected. For example, by iteratively updating Knowledge Graph **225**, Matching and Reconciliation Engine **120** employs CSD to detect the discrepancy, both while processing the ASN and later while processing the invoice with an updated Knowledge Graph **225**.

[0261] Moreover, Anomaly Detector **235** and Anomaly Resolver **245** both have opportunities to apply a different type of training to detect and resolve anomalies (focused specifically on anomalies within and across documents, as opposed to common matching and reconciliation tasks—also within and across documents). The integration with Narrative Generator **250** also affords system **200** with opportunities to generate and target specific notifications and recommendations (all based on an iteratively updated Knowledge Graph **225**) to the appropriate users, in accordance with a scenario-specific workflow.

[0262] Finally, Decisioning Engine **275** affords yet another opportunity to “close the loop” of resolving an anomaly such as this pricing discrepancy, but targeted to the specific context of key decisions (e.g., paying an invoice). In this manner, the various modules of system **200** work together in an integrated fashion on an event-driven asynchronous basis continuously over time, controlled by Scenario Controller **202** in accordance with a scenario-specific workflow.

SCENARIO 3 ANOMALY—Volume Discount Discrepancies between Contract and Invoice

[0263] This scenario involves the same personnel from ACME and ACE, but in the context of common discrepancies relating to volume discounts. In this scenario, described in Table 4 below, ACME and ACE enter into a contract, including a volume discount schedule, showing pricing of \$30/unit for 1-10 units, \$25/unit for 12-25 units and \$20/unit for more than 25 units. This pricing is based on the total of orders under a single monthly invoice.

[0264] Alice issues a PO to Steve at ACE for 8 units at \$30 each. Frank receives an ASN showing that same quantity of 8 units at \$30 each. Shortly thereafter, Alice emails Steve with a modification to the PO, adding 4 more units for a total of 12 units. Her email references the volume discount schedule. Steve promptly confirms the modified PO in a return email to Alice.

[0265] Soon after, Frank receives an ASN from ACE showing the additional 4 units at \$30 each. The ASN does not reference the emails between Alice and Steve.

[0266] In this embodiment, while processing the ASN, Matching and Reconciliation Engine **120** does not detect a pricing discrepancy with the \$30 unit price in the ASN (as the PO also included a \$30 unit price), but it does detect that the additional 4 units do not seem to be reflected in the original PO (and no modified PO has been processed). In other embodiments, Matching and Reconciliation Engine **120** would detect that the PO had been modified (in emails between Alice

and Steve) and that pricing had also been modified.

[0267] Yet, in this embodiment, no anomaly is detected until the subsequent emails between Steve and Alice have been processed, and Knowledge Graph **225** has been updated accordingly. Upon traversing the updated Knowledge Graph **225**, Anomaly Detector **235** detects the pricing discrepancy between the ASN (and original PO) and the updated PO.

[0268] Having been trained on interpreting and analyzing contractual provisions, and similar “patterns” of such provisions conflicting with emails, POs, invoices and other documents, Anomaly Detector **235** not only detects this pricing anomaly, but also detects the potential relevance of the key contractual provisions (i.e., the volume discount schedule) and the PO, ASN and emails modifying the PO. All of this information is reflected in its updates to Knowledge Graph **225**.

[0269] In a subsequent iteration, Anomaly Resolver **245** determines that the volume discounted unit price of \$25 prevails for all 12 units, and addresses the anomaly by notifying Sam of this discrepancy. At this point, Sam elects to notify ACE of this error in an email indicating that all 12 units should reflect the discounted \$25 price.

[0270] Nevertheless, Sam eventually receives the invoice for \$360 (12 units at \$30 each), and again fails to remember receiving the prior notification from system **200** of this volume pricing discrepancy, or his follow-up email to ACE. Once again, in this embodiment, while processing the invoice, Matching and Reconciliation Engine **120** fails to detect this discrepancy between the \$30 unit pricing and the volume discount pricing reflected in the emails between Alice and Steve and in the contract. Nevertheless it updates Knowledge Graph **225** accordingly.

[0271] Fortunately, Anomaly Detector **235** confirms detection of this volume discount pricing anomaly as well as the relevance of the emails and the volume discount schedule in the contract. As was the case in the prior scenario, the prior updating of Knowledge Graph **225** increases the confidence level of this result. Similarly, in a subsequent iteration, Anomaly Resolver **245** reconfirms that the discounted unit price of \$25 prevails for all 12 units (also with greater confidence).

[0272] Recognizing that system **200** had already notified Sam, and that Sam had emailed ACE, Anomaly Resolver **245** now resolves the anomaly by making a dual recommendation—i.e., (i) that Sam be notified again of the pricing discrepancy with an updated narrative, but also (ii) that the invoice be disputed due to the \$60 overpricing. Anomaly Resolver **245** also updates Knowledge Graph **225** accordingly.

[0273] As a result, Narrative Generator **250** is invoked to generate an updated narrative for Sam, which is delivered to Sam (during a subsequent iteration) by Scenario Controller **202** via External Communicator **295**. In addition, Scenario Controller **202** issues a Decision event **276** and invokes Decisioning Engine **275**, which confirms the recommendation to Sam that the invoice be disputed (again employing Narrative Generator **250** to generate the updated narrative containing this recommendation). Sam then proceeds to dispute the invoice with ACE, resulting in a payment of \$300 per the volume discount schedule, as opposed to the erroneous \$360 invoice (inflated due to an error in failing to implement the volume discount once the order had been modified from 8 to 12 units).

TABLE-US-00004 TABLE 4 HUMAN ACTIVITIES DOCUMENT COMMENTS ACE and ACME execute Contract CONTRACT Volume Discount Schedule - based on Invoice \$30/unit for 1-10 units \$25/unit for 12-25 units \$20/unit for more than 25 units Alice issues PO to Steve at ACE PO Quantity of 8 units at \$30 each ASN from ACE arrives to Frank ASN Shows Quantity of 8 units at \$30 each Alice email to Steve Email Modifying PO - adds 4 more units (12 total) References Volume Discount Schedule Steve email to Alice Email Confirms Modified PO ASN from ACE arrives to Frank Email Error in ASN - shows 4 units at \$30 each Sam email to ACE Email Error in ASN - all 12 units should be \$25 each Sam receives Invoice Invoice \$360 (12 × \$30) - no mention of error

[0274] Note that, in other circumstances, the orders for 8 units and 4 units could be considered



distinct orders to be invoiced separately—both at the \$30 non-discounted pricing in accordance with the volume discount schedule. In short, the result is dependent upon context, including the timing of the orders.

[0275] In this scenario, the emails between Alice and Steve provided the key context that enabled system **200** to infer that the discounted unit price of \$25 applied (per the volume discount schedule in the contract). In other words, system **200** extracted this context and derived meaning from it in reaching the conclusion that the \$30 unit pricing in the invoice was incorrect.

[0276] These relationships (between POs, ASNs and invoices on one hand, and the emails and discounted pricing schedule on the other hand) illustrate the concept of DDRs that are even more difficult to discern than those in the prior scenario, in which there was a more explicit discrepancy among the typical documents (i.e., the \$10 unit price in the quote and PO, as compared with the \$11 unit price in the ASN and invoice).

[0277] By making these inferences automatically, system **200** avoids the need for human intervention, which may or may not have resulted in detection of this anomaly, much less automatic correction of the anomaly via prompt natural-language notifications and recommendations to the relevant personnel. Moreover, this scenario further illustrates why it is not feasible for humans to identify the myriad of different types of anomalies and variations thereof and program “rules” to resolve them.

#### SCENARIO 4 Interactive Document Processing

[0278] Turning to FIG. 4, screenshot **400** illustrates an embodiment of an interactive session of the present invention in which an AP clerk submits a recent invoice to system **200** for processing. In this scenario, system **200** not only processes the invoice in the manner illustrated above with respect to other scenarios, but also engages in an interactive natural-language conversation with the AP clerk regarding various aspects of the overall scenario (relying on Interrogation Engine **290**, among other modules).

[0279] It should be noted that, prior to processing this invoice, system **200** has the benefit of an iteratively updated Knowledge Graph **225**, which essentially contains the current state of previously processed documents and information from related transactions, such as other payables, bank account balances, etc. In short, this scenario illustrates an automated alternative to the AP clerk having to manually craft and interpret the results of multiple independent structured database queries, or engage in email or telephone conversations with other personnel. In other scenarios, employees throughout a company can engage in similar interactive communications with system **200**, and thus avoid having to contact AP clerks or other staff members.

[0280] The AP clerk in this scenario simply submits the invoice to system **200** along with a natural-language request to process that invoice. As system **200** proceeds to capture and classify the invoice (displaying an animation illustrating that process), it provides almost immediate results, including the amount of the invoice and vendor name, as well as key summary information.

[0281] For example, it indicates (in the form of two checkboxes with checkmarks) that (1) the invoice total falls below the maximum approval limits, letting the AP clerk know that no additional approvals are required in accordance with the scenario-specific workflow; and (2) the risk and fraud scores both fall below predefined thresholds, letting the AP clerk know that no exception-processing is necessary in accordance with the scenario-specific workflow. It also asks the AP clerk whether it would like to see additional details beyond this key summary information, and whether it would like to continue the conversation.

[0282] In response, system **200** notes a “new” item description—i.e., one with which it is not familiar, perhaps because it did not see a matching description on a prior PO or other document. It includes the description itself (“AT&T 50mB DIA circuit”), along with its line number, quantity, unit price and total price.

[0283] System **200** also asks whether this line item should be included in the “approver” report (presumably because it is unusual, and might be of interest to the person approving the invoice).

The AP clerk responds “Yes” (to instruct system **200** to include that line item in the approver report), to which system **200** responds by noting that the other data in the invoice (taxes, total, payment, etc.) are consistent with prior invoices.

[0284] System **200** then proactively asks whether it should “reconcile” the invoice. While it might invoke Matching and Reconciliation Engine **120** in any event, it is essentially asking the AP clerk whether they want to see the results and continue the conversation. Upon getting an affirmative response, system **200** displays an animation of the reconciliation process and then promptly notes that the line-item reconciliation matches a particular PO (**#1280**), and that all items are account coded. This provides the AP clerk with an immediate indication that no unmatched line items or other anomalies were found. Otherwise, system **200** would have brought those to the attention of the AP clerk.

[0285] System **200** then proactively notes that this invoice is due in 30 days, but that another invoice is 45 days past due and asks whether it should be coded for “priority” payment. Upon getting an affirmative response, system **200** responds by indicating that the invoice has been forwarded for payment to the designated approver (again in accordance with the scenario-specific workflow) and that additional details are available in the approver's report.

[0286] Upon receiving a “Thanks” from the AP clerk, note that system **200** essentially pauses the conversation, as it has no additional questions or other information to add to the conversation at the present time. Yet, system **200** proceeds with its continuous event-driven process, tracking the passage of time and handling any other events that may arise (even if not reflected in this particular conversation).

[0287] After 3 more minutes have elapsed, the AP clerk continues the conversation by requesting that system **200** generate a month-end report for all invoices approved for payment. Note that this request (like other communications from the AP clerk and others) is received (via External Communicator **195**) by Scenario Controller **202**, which generates a User Query event **291** to invoke Interrogation Engine **290**. Because Knowledge Graph **225** has been iteratively updated in the interim, the full transcript of the conversation appears as if nothing has changed, with the exception of a new timestamped entry including the natural-language text of the AP clerk's request.

[0288] In response, system **200** (with the assistance of Interrogation Engine **290** and Narrative Generator **250**, among other modules) promptly indicates that the report is ready to be downloaded (in XL format) and asks whether the AP clerk would like to save the template. Upon receiving an affirmative response, the report is promptly downloaded.

[0289] But system **200** also proactively notes that there is an outstanding balance (\$123) and that a designated amount (not shown) will be needed to satisfy next week's payable obligations. In response, the AP clerk requests that system **200** pay a designated vendor that amount (based upon the bank account information on record).

[0290] Note that the AP clerk need not remember the bank account information or other related information, as this information is known to system **200** via iteratively updated Knowledge Graph **225**. As a result, system **200** has all of the information it needs to forward this request along with all relevant information to the relevant personnel who will implement the payment. This process (not shown) occurs promptly, but may require multiple iterations involving various modules, including Interrogation Engine **290**, Narrative Generator **250** and Scenario Controller **202**, each of which updates Knowledge Graph **225** over time.

[0291] In this scenario, the AP clerk also requests that system **200** pay a different vendor the amount due (per a specified invoice number) by issuing a one-time “virtual card.” The reason for this “real time” request is that system **200** just informed the AP clerk of the total amount of payables due next week. So, the AP clerk generated this request to avoid further draining the company's primary bank account given the amount of payables due next week.

[0292] System **200** promptly informs the AP clerk that payments are now scheduled for the two specified invoices on designated dates, and provides the AP clerk a link to see payment status for

all invoices. In addition, system **200** offers to email the two vendors the company's standard advance payment notice (to their email addresses on record), to which the AP clerk responds affirmatively.

[0293] Finally, system **200** proactively notifies the AP clerk that, while \$10,456 in payables is due next week, the default account only has a current balance of \$9780. It therefore asks whether it should transfer funds, to which the AP clerk responds affirmatively. Note that this natural-language conversation leaves out many (i.e., unnecessary) details that are known to system **200** via Knowledge Graph **225** (e.g., the bank account from which it will transfer funds to refresh the balance of the default account).

[0294] In this manner, system **200** eliminates a great deal of human intervention that would otherwise be required to perform these various tasks, including spending a significant amount of time and expense manually generating database queries and engaging in various email and telephone conversations with other personnel. Note also that system **200** not only detects and responds to events, but also anticipates the occurrence of future events and hypothesizes the potential occurrence of other events.

[0295] It will be apparent to one skilled in the art from this and other scenarios that the integration of key components of system **200** enables a continuous event-driven system that detects and responds to new events as new information is obtained (including the mere passage of time) by leveraging the iteratively updated Knowledge Graph **225** to perform lower-level document processing tasks (e.g., capture, classification, matching, reconciliation, verification, compliance and coding) as well as higher-level tasks such as decisioning, analytics and interrogation. Without such integration, many of these tasks could not be automated, which would result in even more frequent need for human judgment and intervention.

#### MISCELLANEOUS CONCEPTS Across Scenarios

[0296] As noted above, the concept of the passage of time as an event is a significant aspect of the present invention in virtually any scenario. For example, system **200** captures the “due date” of an invoice directly from the document, or perhaps indirectly from other information in that or another document (such as a standard term in a contract or PO stating that invoices are due 30 days from issuance). By tracking time while performing other event-driven tasks, system **200** detects when the due date of the invoice is approaching (e.g., 5 days in advance per a predefined configuration or scenario-specific workflow) and notifies appropriate personnel if that invoice remains unpaid.

[0297] Moreover, in the event the value of a date (and/or time) field of an invoice was the result of a capture, data entry or other error, such error may go undetected for a period of time—e.g., preventing the invoice from being properly matched to its corresponding PO. Yet, as more documents are processed, system **200** gathers greater evidence that the value of this date field is an anomaly that can be corrected—perhaps by reinvoking Matching and Reconciliation Engine **120**. The correction of this error will then enable system **200** to determine when the due date of the invoice is approaching (or, in some cases, that the due date has already passed).

[0298] In other scenarios, as alluded to above, a “document” may include an audio recording or other file reflecting a phone conversation between multiple people, such as representatives of a company and one of its suppliers regarding an overdue invoice. As noted above, the processing of that document by a TSAgent **210** may result in the capture of metadata relating to the tone of voice of the supplier's representative, which may (in conjunction with other factors relating, for example, to the relative importance of the supplier) escalate the need to pay the invoice promptly.

[0299] The importance of context cannot be overemphasized with respect to the present invention. As a simple example, a heavy coffee stain on a paper receipt may prevent many fields of the receipt from being captured properly (e.g., by OCR), and may even prevent proper classification of the receipt (as it may be difficult to distinguish from other documents). As a result, an employee's subsequent submission of a reimbursement form might raise an issue due to the lack of proper receipts.

[0300] Because system **200** performs its tasks in an integrated manner across documents over time, it may initially capture the date and other fields of the “unknown” receipt, but only later process the reimbursement form, and match sufficient fields to warrant reinvocation of Matching and Reconciliation Engine **120**, ultimately resulting in a proper classification of the receipt which allows the employee to be reimbursed without delay.

[0301] Still other scenarios involve audits for fraud detection, the calculation of rebates and other processes that are typically performed only after relevant transactions have been completed, and which often require significant manual labor at great cost of both time and money. System **200** facilitates the performance of such tasks on a continuous basis over time, which facilitates the detection and resolution of relevant anomalies as they occur.

[0302] For example, an audit for fraud detection may be performed as each document is submitted for processing. The results will be negative for many iterations unless and until a particular pattern is detected, suggesting possible fraud. Even if that anomaly cannot be resolved at the time it is first detected, relevant personnel will be notified, enabling them to implement additional safeguards to prevent any potential problem from being exacerbated.

[0303] Moreover, consider a rebate scenario in which a supplier may owe rebates to a company, but such rebates can only be calculated at the end of a calendar quarter after all relevant transactions have been completed. By implementing this rebate calculation on a continuous basis over time, system **200** is able to provide advance notification to relevant personnel. For example, weeks before the end of the calendar quarter, system **200** may determine that rebates are beginning to accumulate. Upon receiving this early notification from system **200**, the relevant company personnel may decide to alter its purchasing practices (e.g., to maximize rebates from a supplier). From a supplier's perspectives, the obligation of paying rebates will at least not be a last-minute surprise (e.g., months later when the supplier receives a notification from the company that a large rebate is due).

[0304] As alluded to above, even within the financial realm, a vast array of scenarios are accommodated by the present invention. For example, the wholesale or retail lockbox scenarios are prime candidates for the enhanced automation offered by system **200**. The sheer number of errors in these scenarios is enormous. The text on invoices, checks, envelopes, remittance and other documents is prone to handwriting as well as data entry errors, and the need to match documents across thousands or millions of distinct transactions is significant.

[0305] Reconciling, matching and verifying data across ACH ledgers and addenda records (e.g., regarding payors and payees, dates, amounts and miscellaneous notes) requires a significant reliance on context across documents to reach accurate conclusions. System **200** avoids much of the significant manual labor currently required to perform the tasks of detecting and correcting these errors by automatically identifying and extracting relevant relationships over time (stored in iteratively updated Knowledge Graph **225**), and reinvoking document-processing TSAgents **210** as new documents and other information are obtained (often out of chronological order).

[0306] Apart from these scenarios, system **200** is applicable to auto purchases, loans, construction and many other fields. A vast array of common types of anomalies detected and resolved by system **200** include pricing anomalies (errors and changes in amounts, overpricing, unauthorized discounts and surcharges, etc.), quantity anomalies (unauthorized over/under ordering, mismatches between ordered and received items, etc.), fraud (ghost vendors and other forms of vendor fraud, unauthorized vendor preferences affecting pricing and quality of goods and services, employee collusion with vendors to inflate prices and generate fraudulent POs, etc.), contractual terms (re delivery, payment terms, discounts, service levels, etc.), invoice and payment discrepancies (duplicate invoices, misallocation of payments, delays, etc.), approvals (missing approvals, unauthorized purchases beyond approved limits, etc.), compliance and regulation (violating legal and internal rules, inadequate audit trail documentation, etc.), data entry and system errors (re price, quantity, vendor details, inconsistencies across systems, etc.), market conditions (unexpected

changes such as sudden price increases, shortages, etc.) and supply chain issues (e.g., due to natural disasters, political instability, transportation disruptions, etc.).

[0307] Even discrepancies in invoices alone are too numerous to address via preprogrammed rules. Invoice-related discrepancies affect vendor info, pricing, quantities, shipping charges, taxes, currency, delivery dates, volume and other discounts, damaged goods, contractual SLAs, etc.

[0308] In short, there is simply no human substitute for training models on the myriad of “patterns” of common discrepancies and other errors that occur across a wide range of scenarios, both within and outside the financial realm. Simply employing machine-learning models is helpful, but insufficient without a highly integrated continuous event-driven system that iteratively derives meaning from context.

[0309] As noted above, a vast array of other scenarios (in which the combination of TSAgents 210 with an iteratively updated Knowledge Graph 225 enables a significant reduction in the need for human judgment and intervention) will become apparent without departing from the principles of the present invention.

[0310] The present invention has been described herein with reference to specific embodiments as illustrated in the accompanying drawings. It should be understood that, in light of the present disclosure, additional embodiments of the concepts disclosed herein may be envisioned and implemented within the scope of the present invention by those skilled in the art.

## Claims

1. A method for deriving meaning from the context of data within and across a plurality of documents, the method comprising the following steps: (a) capturing data from each of the plurality of documents; (b) deriving a plurality of relationships among the captured data; (c) employing contextual semantic derivation to derive one or more such relationships that are difficult to discern from the captured data without human intervention; (d) generating and updating a knowledge graph reflecting the plurality of derived relationships; and (e) traversing the knowledge graph in response to one or more internally or externally generated events, including the receipt of a document, the passage of time or the detection of an anomaly, wherein the traversal of the knowledge graph facilitates the performance of one or more tasks dependent upon at least one of the difficult-to-discern relationships.
2. An iterative event-driven system that derives meaning from the context of data within and across a plurality of documents, the system comprising: (a) one or more task-specific agents that perform document-processing tasks in response to a new document event, wherein each task-specific agent employs contextual semantic derivation to derive one or more relationships among data across the new document and one or more of the plurality of documents previously processed by the system, and wherein at least one of such derived relationships is difficult to discern from the plurality of documents without human intervention; (b) a knowledge graph reflecting the relationships derived by the one or more task-specific agents; and (c) an anomaly detector that traverses the knowledge graph to detect an anomaly with respect to the one or more difficult-to-discern derived relationships.
3. A self-training system that enhances the training of a primary trained model based on actual production data that yielded an insufficient confidence level when presented to the primary trained model, the system comprising: (a) the primary trained model which, when presented with actual production data within a target document, outputs a result among a plurality of potential results that yields the highest confidence level, provided that the highest confidence level exceeds a predefined threshold; (b) a plurality of secondary trained models; (c) a validation engine that is invoked when the primary trained model is unable to output a result with a confidence level exceeding the predefined threshold, wherein the validation engine (i) performs multi-model voting by submitting the actual production data within the target document to the plurality of secondary trained models

and selecting a winning result based on a majority vote among the secondary trained models; (ii) submits the selected winning result to the system as if the primary trained model had originally output that result; and (ii) generates a template from the actual production data for use in retraining the primary trained model for future use; and (d) a model retrainer that enhances the training of the primary trained model by using the template generated by the validation engine.

---